

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Γραπτές εξετάσεις: 25 Ιανουαρίου 2023 (90 λεπτά)

Θέμα Α (2 μονάδες)

1. Τι είναι ο Πολυμορφισμός; Ποιες υποκατηγορίες περιλαμβάνει ο Πολυμορφισμός;
2. Ποιοι είναι οι άλλοι τρεις βασικοί πυλώνες του Αντικειμενοστραφούς Προγραμματισμού;

Θέμα Β (4 μονάδες)

1. Περιγράψτε με λόγια τι κάνει ο παρακάτω κώδικας (C++)
2. Δώστε το αποτέλεσμα της εκτέλεσής του.

```
01      #include <iostream>
02      using namespace std;
03
04      class B1 {
05      public:
06          void f1() { cout << "B1 f1()\n"; }
07          virtual void f2() { cout << "B1 f2()\n"; }
08          virtual void f3() { cout << "B1 f3()\n"; }
09          virtual void f4() { cout << "B1 f4()\n"; }
10      };
11
12      class B2 : public B1 {
13      private:
14          string nickname;
15      public:
16          B2(string input) : nickname(input) {};
17          void f1() { cout << nickname << " f1()\n"; }
18          void f2() { cout << nickname << " f2()\n"; }
19          void f4(int a = 0) { cout << a << " from " << nickname << " f4()\n"; }
20      };
21
22      void order(B1& in) { in.f2(); }
23
24      int main() {
25          B1* parent_obj;
26          B2 kid01("NKUA");
27          B2 kid02("AUEB");
28          B2 kid03("Ionian");
29
30          parent_obj = &kid01;
31          parent_obj->f1();
32          parent_obj = &kid02;
33          parent_obj->f2();
34          parent_obj = &kid03;
35          parent_obj->f3();
36          parent_obj->f4();
37          order(kid01);
38          return 0;
39      }
```

3. Στον παρακάτω κώδικα (C++) ο compiler εντοπίζει ένα λάθος. Εξηγήστε γιατί.
4. Αφότου το διορθώσετε, δώστε το αποτέλεσμα της εκτέλεσης του προγράμματος.

```

01    #include <iostream>
02    using namespace std;
03
04    class street_food {
05    private:
06        string food_name;
07    public:
08        void set_food_name(string n) { food_name = n; }
09    };
10
11    class pitogyro : public street_food {
12    private:
13        bool exe_i_tzatziki;
14    public:
15        void set_exe_i_tzatziki(bool n) {
16            exe_i_tzatziki = n;
17            cout << food_name << " set the tzatziki status\n";
18        }
19    };
20
21    int main(){
22        pitogyro mam;
23        mam.set_food_name("souvlaki me patates");
24        mam.set_exe_i_tzatziki(true);
25        return 0;
26    }

```

Θέμα Γ (4 μονάδες)

Έχουμε ένα μέρος από το παραμύθι «Ο Καλός Λύκος και τα Γουρουνάκια»:

1. Ο Καλός Λύκος έχει όνομα, ηλικία και μπορεί να βρίσκεται σε οποιαδήποτε θέση (ακέραιος αριθμός) μέσα στον μονοδιάστατο κόσμο του παραμυθιού. Μπορεί να μιλήσει/ρωτήσει (εκτύπωση μηνύματος στην οθόνη), να μετακινηθεί (αύξηση/μείωση θέσης) και να φουσήξει (γκρεμίζοντας έτσι κάποιο σπίτι).
2. Το κάθε Γουρουνάκι έχει όνομα, ηλικία, μπορεί να βρίσκεται σε οποιαδήποτε θέση (ακέραιος αριθμός) μέσα στον μονοδιάστατο κόσμο του παραμυθιού. Επίσης, μπορεί να μιλήσει (εκτύπωση μηνύματος στην οθόνη) και να μετακινηθεί μέσα σε αυτόν (αύξηση/μείωση θέσης), αλλά πιο γρήγορα από τον Καλό Λύκο.
3. Υπάρχουν Σπίτια, το καθένα με τα εξής χαρακτηριστικά: υλικό δόμησης (string), γκρεμισμένο ή γερό, θέση στον μονοδιάστατο κόσμο του παραμυθιού.
4. Εάν ο Καλός Λύκος, κάποιο Σπίτι και κάποιο Γουρουνάκι βρεθούν στο ίδιο σημείο, μέσα στον μονοδιάστατο κόσμο του παραμυθιού, τότε ο Καλός Λύκος ρωτάει αν μπορεί να μπει στο σπίτι (έξοδος στην οθόνη). Αν η απάντηση (είσοδος από το πληκτρολόγιο) είναι Όχι, τότε ο Καλός Λύκος φουσάει (γκρεμίζοντας το σπίτι) και το Γουρουνάκι μετακινείται. Ορίστε την κατάλληλη συνάρτηση.

Υλοποιήστε σε C++ τις κλάσεις με τα αντίστοιχα μέλη, καθώς και όποια/όποιες άλλες μεταβλητές ή συναρτήσεις χρειάζονται εντός ή εκτός των κλάσεων, με βάση τις αρχές του Αντικειμενοστραφούς Προγραμματισμού.

Επισημάνσεις:

- Τα χαρακτηριστικά/δεδομένα (attributes) των κλάσεων δεν θα πρέπει να είναι δημόσια.
- Δεν σας ζητείται να υλοποιήσετε τη main().
- Μπορείτε να κάνετε τις δικές σας παραδοχές για την υλοποίηση, αρκεί να μην αναιρούν την παραπάνω περιγραφή.

Καλή επιτυχία!!

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Εξετάσεις 20 Σεπτεμβρίου 2016

1. (α') Γράψτε μια παράγραφο περίπου 30 λέξεων που να περιέχει τις λέξεις “αντικείμενο”, “κλάση”, “κληρονομικότητα” και “αρχικοποίηση” στο νοηματικό πλαίσιο του αντικειμενοστραφούς προγραμματισμού.

- (β') Δίδεται το παρακάτω πρόγραμμα C++:

```
#include<iostream>
using namespace std;

class A{ int data;
public:
A(int i=10):data(i) { cout << "I am constructing an A with " << data << endl; }
~A(){ cout << "I am deleting an A with " << data << endl; }

// virtual void bla() {cout << "This is A::bla" << endl; data++; }
// void bla() { cout << "This is A::bla" << endl; data++; }
// virtual void bloo(){ cout << "This is A:bloo()" << endl; bla(); }
// void bloo(){ cout << "This is A::bloo()" << endl; bla(); }
};

class B : public A { int data;
public:
B(int i=100):data(i){ cout << "I am constructing a B with " << data << endl; }
~B(){ cout << "I am deleting a B with " << data << endl; }

void bla() { cout << "This is B::bla" << endl; data++; }
void bloo(){ cout << "This is B:bloo()" << endl; bla(); } };

int main(){
A a; B b; A* pa;
pa = &b;
pa->bla(); pa->bloo(); }
```

Αποσχοιάζοντας δύο από τις παραπάνω σχολιασμένες γραμμές τη φορά ώστε το πρόγραμμα να μεταγλωττίζεται, δώστε το αποτέλεσμα της εκτέλεσης, αιτιολογώντας την απάντησή σας. Αυτό να γίνει για όλους τους δυνατούς συνδυασμούς αποσχολιασμού.

2. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++:

```
#include <iostream>
using namespace std;

class A{ int data;
public:
    A(int di=10): data(di)
        { cout << "I just created an A with " << data << endl; }
    A(const A& a): data(a.data)
        { cout << "I just created an A by Copying" << endl; }
    ~A(){ cout << "I shall delete an A with " << data << endl; }

    void inc() { data++; } };

class B{ A data1; A* data2;
public:
    B(const A& a): data1(a), data2(NULL)
        { cout << "I just created a B " << endl; }
    B(const B& b): data1(b.data1), data2(b.data2)
        { cout << "I just created a B by Copying" << endl; }
    ~B(){ cout << "I shall delete a B " << endl; }

    void inc() { data1.inc(); if (data2 != NULL) data2->inc(); }
    void set(A* pa) { data2 = pa; } };

class C{ B data1; B& data2;
public:
    C(B& b): data1(b), data2(b)
        { cout << "I just created a C " << endl; }
    C(const C& c): data1(c.data1), data2(c.data2)
        { cout << "I just created a C by Copying" << endl; }
    ~C(){ cout << "I shall delete a C " << endl; }

    void inc() { data1.inc(); data2.inc(); } };

class D{ C& data1; C& data2;
public:
    D(C& c1, C& c2): data1(c1), data2(c2)
        { cout << "I just created a D " << endl; }
    ~D(){ cout << "I shall delete a D " << endl; }

    void inc() { data1.inc(); data2.inc(); } };

int main(){
    A a; B b1(a); B b2(b1); C c1(b1); C c2(c1); D d(c1,c2);
    a.inc(); b1.inc(); b2.inc(); c1.inc(); c2.inc(); d.inc();
    b1.set(&a);
    a.inc(); b1.inc(); b2.inc(); c1.inc(); c2.inc(); d.inc(); }
```


3. Υλοποιήστε σε C++ την προσομοίωση της υπηρεσίας διαλογής ενός κέντρου διανομής Ταχυδρομείου από το οποίο διανέμονται γράμματα και δέματα.

Κάθε δέμα και κάθε γράμμα έχει μια διεύθυνση παραλήπτη. Αυτή αποτελείται από το δρόμο και τον αριθμό. Τα γράμματα ενδέχεται να είναι συστημένα ή επείγοντα. Τα δέματα ενδεχόμενα χαρακτηρίζονται ως ογκώδη οπότε αντί για το ίδιο το δέμα, διανέμεται ένα απλό γράμμα που έχει τη διεύθυνση του δέματος.

Κάθε τι που διανέμεται χαρακτηρίζεται από μονάδες διανομής. Κάθε απλό γράμμα έχει μία μονάδα διανομής. Ένα συστημένο γράμμα ή ένα επείγον γράμμα έχει δύο μονάδες διανομής. Ένα μη ογκώδες δέμα έχει τρεις μονάδες διανομής.

Τα γράμματα και τα δέματα που πρέπει να διανεμηθούν, καθώς αφικνούνται στο ταχυδρομείο, προστίθενται σε μια λίστα διανομής. Τα επείγοντα γράμματα, όμως, φυλάσσονται σε άλλη λίστα διανομής.

Το κέντρο αρχικά δεν έχει τίποτα να διανείμει. Στο κέντρο προστίθεται ένα γράμμα για διανομή (`arrive`), τοποθετώντας το στην κατάλληλη λίστα για διανομή. Στο κέντρο προστίθεται ένα δέμα για διανομή (`arrive`), προωθώντας το κατάλληλα για διανομή. Το κέντρο χαρακτηρίζεται από το πλήθος των μονάδων διανομής των αντικειμένων που οφείλει να διανέμει καθώς και το πλήθος των ογκωδών δεμάτων που φυλάσσει για παράδοση.

Ένα γράμμα αρχικοποιείται, αναθέτοντας τη διεύθυνση παραλήπτη.

Ένα δέμα αρχικοποιείται, αναθέτοντας τη διεύθυνση παραλήπτη.

Υλοποιήστε τα παραπάνω μέσω καταλλήλων κλάσεων, ορίζοντας μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά.

Όπου θεωρείτε απαραίτητο, κάνετε παραδοχές στις προδιαγραφές, τεκμηριώνοντάς τις.

Σημείωση: Θεωρήστε δεδομένο έναν τύπο λίστας με τις λειτουργίες που χρειάζεστε.

**ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ**

Εξετάσεις 21 Σεπτεμβρίου 2017

1. (α') Δώστε μια ομοιότητα και μια διαφορά μεταξύ της C++ και της Java όσον αφορά ορισμό κλάσεων και μια ομοιότητα και μια διαφορά όσον αφορά ορισμό αντικειμένων.
(β') Δίδεται το παρακάτω πρόγραμμα Java:

```
public class MainClass {
    public static void main(String[] args) {
        A a1 = new A(); A a2 = new A(10);
        B b1 = new B(); B b2 = new B(100);
        System.out.println("////////////////////////////////////////");
        a1.print(); a2.print();
        b1.print(); b2.print();
        System.out.println("////////////////////////////////////////");
        a1 = b1; a1.print(); } }

class A{
    private int data;
    { System.out.println("A new A was just created"); }
    public A() {
        System.out.println("The A data is " + data); }
    public A(int d) { data = d;
        System.out.println("The A data is " + data); }
    public void print() { System.out.println("The A data is " + data); } }

class B extends A{
    private int data;
    { System.out.println("A new B was just created"); }
    public B() { System.out.println("The B data is " + data); }
    public B(int d) { super(d); data = d;
        System.out.println("The B data is " + data); }
    public void print() { super.print();
        System.out.println("The B data is " + data); } }
```

Επίσης, δίδεται και το παρακάτω πρόγραμμα C++:

```
#include <iostream>
using namespace std;

class A{ int data;
public:
    A(int d = 0) {
        cout << "A new A was just created" << endl;
        data = d;
        cout << "The A data is " << data << endl ; }
    virtual void print() { cout << "The A data is " << data << endl ; } };
```

```

class B : public A{ int data;
public:
    B(int d = 0) : A(d) {
        cout << "A new B was just created" << endl;
        data = d;
        cout << "The B data is " << data << endl ;}
    void print() {
        A::print();
        cout << "The B data is " << data << endl; } };

int main() { A a1; A a2(10); B b1; B b2(100);
cout << "/////////////////////////////////" << endl;
a1.print(); a2.print(); b1.print(); b2.print();
cout << "/////////////////////////////////" << endl;
a1 = b1; a1.print(); }

```

Δώστε τα αποτελέσματα του καθενός, αιτιολογώντας την απάντησή σας. Που διαφέρουν και γιατί;

2. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++:

```

#include<iostream>
using namespace std;

class Block{ int data;
public:
    Block(int datav = 0) : data(datav)
        { cout << "Constructing a Block with " << datav << endl; }
    ~Block(){ cout << "Destructing a Block with: " << data << endl; }
    void double_it() { data = data * 2 ; }
    void print() { cout << data << endl; } };

class B{ Block data1; Block* data2; Block* data3;
public:
    B() : data2(NULL),data3(NULL) { cout << "Constructing a B" << endl; }
    B(int value1, Block* value2) : data1(value1)
        { cout << "Constructing a B with given values " << endl;
          data2 = new Block(value1); data3 = value2; }
    ~B(){ cout << "Destructing a B " << endl;
        if (data2 == NULL) cout << "with NULL data2" << endl;
        else { cout << "with data2: " << endl; delete data2; }
        if (data3 == NULL) cout << "with NULL data3" << endl;
        else{ cout << "with data3: " << endl; data3->print(); }}

    void double_it(){data1.double_it(); data2->double_it(); data3->double_it();}
    void double_data1(){ data1.double_it(); }
    void double_data2(){ data2->double_it(); }
    void double_data3(){ data3->double_it(); } };

int main(){ Block block(20); B b(10,&block);
    block.double_it(); b.double_it(); b.double_data1();
    b.double_data2(); b.double_data3(); block.double_it(); }

```

3. Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση ενός ταχυφαγείου. Το ταχυφαγείο αποτελείται από ένα σύνολο από “Θέσεις εξυπηρέτησης”. Οι πελάτες που εισέρχονται στο ταχυφαγείο εξυπηρετούνται ένας-ένας από τις “Θέσεις Εξυπηρέτησης” στις οποίες σχηματίζουν ουρές. Όταν εξυπηρετείται ένας πελάτης, δημιουργείται μια παραγγελία. Επίσης, ο πελάτης αφαιρείται από την ουρά της “Θέσης Εξυπηρέτησης” που περίμενε και προστίθεται σε μια λίστα πελατών του ταχυφαγείου σε αναμονή της παράδοσης της παραγγελίας του.

Μια παραγγελία αναφέρεται στον κωδικό του μενού που περιέχει και σχετίζεται με τη “Θέση Εξυπηρέτησης” από την οποία παραγγέλθηκε. Τελικά ανατίθεται σε έναν πελάτη (οποιονδήποτε) που έχει παραγγείλει, από τη “Θέση Εξυπηρέτησης” αυτή, μενού με κωδικό ίδιο με τον κωδικό του μενού της παραγγελίας.

Για την προσομοίωση να χρησιμοποιηθούν οι κλάσεις “Ταχυφαγείο” (`FastFoodRestaurant`), “Θέση εξυπηρέτησης” (`Counter`), “Παραγγελία” (`Order`) και “Πελάτης” (`Customer`).

Η κλάση “Ταχυφαγείο”:

- έχει 5 Θέσεις εξυπηρέτησης (`counters`)
- έχει μια λίστα από πελάτες σε αναμονή της παράδοσης της παραγγελίας τους (`waiting_customers`)

Η κλάση “Ταχυφαγείο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά η λίστα πελατών σε αναμονή είναι κενή
- Γίνεται άφιξη πελάτη, ο οποίος προστίθεται για να περιμένει στην θέση εξυπηρέτησης στην μικρότερη ουρά (`arrive`)
- Γίνεται εξυπηρέτηση, εξυπηρετώντας πελάτη από τη θέση εξυπηρέτησης με τους περισσότερους πελάτες σε αναμονή, δημιουργώντας μια νέα παραγγελία που αντιστοιχεί στην επιλογή του πελάτη αυτού, αναθέτοντάς του την παραγγελία και προσθέτοντας τον πελάτη στη λίστα πελατών εν αναμονή (`serve`)
- Παραδίδεται παραγγελία σε έναν πελάτη από αυτούς εν αναμονή που ταιριάζει η παραγγελία αφαιρώντας τον από τη λίστα πελατών εν αναμονή (`deliver`)

Η κλάση “Θέση εξυπηρέτησης”:

- έχει μια ουρά από πελάτες (`customer_queue`)
- έχει έναν μετρητή των πελατών που περιμένουν στην ουρά (`waiting_customers`)

Η κλάση “Θέση εξυπηρέτησης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά η ουρά των πελατών είναι κενή
- Εξυπηρετεί, αφαιρώντας πελάτη από την ουρά και μειώνοντας τον μετρητή των πελατών που περιμένουν (`serve`)
- Γίνεται άφιξη πελάτη, ο οποίος προστίθεται για να περιμένει στο τέλος της ουράς και αυξάνεται ο μετρητής των πελατών που περιμένουν (`arrive`)

Οι “Παραγγελίες” δομούνται από τον κωδικό του μενού τους και τον αριθμό της “Θέσης εξυπηρέτησης” από την οποία προέκυψαν.

Οι “Πελάτες” δομούνται από τον αριθμό ταυτότητάς τους και την παραγγελία τους. Αρχικά η παραγγελία είναι κενή.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις. Δεν χρειάζεται υλοποίηση συνάρτησης `main`.

Σημείωση 1: Όπου θεωρείτε απαραίτητο, κάνετε παραδοχές στις προδιαγραφές, τεκμηριώνοντάς τις.

Σημείωση 2: Θεωρήστε δεδομένο έναν τύπο λίστας κι έναν τύπο ουράς με τις λειτουργίες που χρειάζεστε.

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Εξετάσεις 9 Φεβρουαρίου 2016

1. (α') Τι εννοούμε με τον όρο "εγκλωβισμός" (encapsulation); Γιατί είναι κρίσιμος για τον αντικειμενοστραφή προγραμματισμό; Δώστε ένα παράδειγμα C++ κώδικα στο οποίο υπάρχει χρήση εγκλωβισμού.

(β') Δίδεται το παρακάτω πρόγραμμα C++:

```
#include<iostream>
using namespace std;

class A{int var;
public:
    A(int i=10):var(i){
        cout << "I just constructed an A with " << var << endl;}
    ~A(){cout << "An A will be destroyed with " << var << endl;}
    void bla(){cout << "I am A:bla() " << endl; var++;}
    void bloo() {cout << "I am A::bloo()" << endl; bla();} };

class B: public A{int var;
public:
    B(int i=100):var(i){
        cout << "I just constructed a B with " << var << endl;}
    ~B(){cout << "A B will be destroyed with " << var << endl;}
    void bla(){cout << "I am B:bla() " << endl; var++;} };

class C: public A{int var;
public:
    C(int i=200):var(i){
        cout << "I just constructed a C with " << var << endl;}
    ~C(){cout << "A C will be destroyed with " << var << endl;}
    virtual void bla(){cout << "I am C:bla() " << endl; var++;} };

class D: public B{int var;
public:
    D(int i=1000):var(i){
        cout << "I just constructed a D with " << var << endl;}
    ~D(){cout << "A D will be destroyed with " << var << endl;} };

class E: public C{int var;
public:
    E(int i=2000):var(i){
        cout << "I just constructed an E with " << var << endl;}
    ~E(){cout << "An E will be destroyed with " << var << endl;}
    void bla(){ cout << "I am E:bla() " << endl; var++;} };

class F: public C{int var;
public:
    F(int i=2222):var(i){
        cout << "I just constructed a F with " << var << endl;}
    ~F(){cout << "A F will be destroyed with " << var << endl;} };
```

```

int main(){ A* pa; B* pb; C* pc; A a; D d; E e; F f;
    a.bla(); a.bloo(); d.bla(); d.bloo();
    e.bla(); e.bloo(); f.bla(); f.bloo();

    pa=&a; pa->bla(); pa->bloo();
    pa=&d; pa->bla(); pa->bloo();
    pa=&e; pa->bla(); pa->bloo();
    pa=&f; pa->bla(); pa->bloo();

    pb=&a; pb->bla(); pb->bloo();
    pb=&d; pb->bla(); pb->bloo();

    pc=&d; pc->bla(); pc->bloo();
    pc=&e; pc->bla(); pc->bloo();
    pc=&f; pc->bla(); pc->bloo(); }

```

Το πρόγραμμα αυτό δεν μεταγλωττίζεται. Εξηγήστε γιατί. Χωρίς να αλλάξετε τους ορισμούς των κλάσεων, σχολιάστε τις ελάχιστες γραμμές ώστε αυτό να μεταγλωττίζεται. Στη συνέχεια, αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του.

- Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++:

```

#include<iostream>
using namespace std;

class A{int i;
public:
    A(int ii=10):i(ii)
        {cout << "I just constructed an A with" << i << endl;}
    A(const A& a):i(a.i)
        {cout << "I just constructed an A by COPYING with "
            << i << endl;}
    ~A() {cout << "I shall destroy an A with " << i << endl;}
    int inc() {return ++i;}
    int get_data() const {return i;} };

class B{A& a;
public:
    B(A& aa):a(aa)
        {cout << "I just constructed a B with "
            << a.get_data() << endl;}
    B(const B& b):a(b.a)
        {cout << "I just constructed a B by COPYING with "
            << a.get_data() << endl;}
    ~B() {cout << "I shall destroy a B" << endl;}
    int inc() {return a.inc();}
    int get_data() const {return a.get_data();} };

class C{ A a; B b;
public:
    C(const A& aa, B& bb):a(aa),b(bb)
        {cout << "I just constructed a C" << endl;}

```

```

    C(const C& c):a(c.a),b(c.b)
        {cout << "I just constructed a C by COPYING" << endl;}
    ~C() {cout << "I shall destroy a C" << endl;}
    void inc() {a.inc(); b.inc();}
    B get_b() {return b;} };

class D{A* a; B b; C& c;
public:
    D(A& aa, const B& bb, C& cc):a(&aa),b(bb),c(cc)
        {cout << "I just constructed a D" << endl;}
    D(const D& d):a(d.a),b(d.b),c(d.c)
        {cout << "I just constructed a D by COPYING" << endl;}
    ~D() {cout << "I shall destroy a D " << endl;}
    void inc() {a->inc(); b.inc(), c.inc();}
    B get_b() {return b;} };

int main(){A a(20); B b(a); C c(a,b); D d(a,b,c);
    a.inc(); b.inc(); c.inc(); d.inc();
    c.get_b().inc(); d.get_b().inc(); }

```

3. Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία απλοϊκή προσομοίωση της εισόδου και εξόδου αυτοκινήτων σε ένα γκαράζ. Στο γκαράζ θεωρούμε ότι σταθμεύουν τόσο (επιβατικά) αυτοκίνητα όσο και φορτηγά, σε τρεις ορόφους. Το γκαράζ έχει μια μέγιστη χωρητικότητα για στάθμευση αυτοκινήτων και μια μέγιστη χωρητικότητα για στάθμευση φορτηγών, ισοκατανεμημένες στους τρεις ορόφους. Προκειμένου να εισέλθει στο γκαράζ ένα αυτοκίνητο ή ένα φορτηγό, προστίθεται το τέλος μιας ουράς εισόδου που οδηγεί στο γκαράζ. Το πρώτο αυτοκίνητο ή φορτηγό της ουράς, αφαιρείται από την ουρά και μπαίνει στο γκαράζ, σταθμεύοντας στον πρώτο όροφο που θα βρεθεί ελεύθερη θέση. Βγαίνει από το γκαράζ το τελευταίο αυτοκίνητο ή φορτηγό (ό,τι είναι αυτό) που στάθμευσε σε έναν όροφο.

Προκειμένου να γίνει η υλοποίηση αυτή θεωρούμε τις κλάσεις: “επιβατικό αυτοκίνητο” (**Car**), “φορτηγό” (**Truck**), “γκαράζ” (**Garage**), “όροφος” (**Floor**) και “ουράΕισόδου” (**EntranceQueue**).

Η κλάση “επιβατικό αυτοκίνητο”:

- έχει έναν όροφο στον οποίο έχει σταθμεύσει (**floor**)

Η κλάση “επιβατικό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, δεν έχει σταθμεύσει σε κάποιον όροφο.
- Το αυτοκίνητο εισέρχεται σε μια ουρά εισόδου γκαράζ, εισάγοντας στην ουρά εισόδου το αυτοκίνητο (**enter**).
- Το αυτοκίνητο σταθμεύει σε έναν όροφο, εάν επιτρέπεται από τον αντίστοιχο μετρητή, αναθέτοντας τον όροφο αυτόν σαν τιμή του ορόφου του και αυξάνοντας τον μετρητή αυτοκινήτων (**park**).
- Το αυτοκίνητο βγαίνει, αναθέτοντας στον όροφό του μηδενική τιμή και μειώνοντας τον μετρητή αυτοκινήτων του ορόφου στον οποίο είχε σταθμεύσει (**exit**).
- Το αυτοκίνητο επιστρέφει τον όροφο στον οποίο έχει σταθμεύσει (**get_floor**).
- Εκτυπώνεται μήνυμα πληρότητας αυτοκινήτων "Car places are full" (**print_wait_message**).

Η κλάση “φορτηγό αυτοκίνητο”:

- έχει έναν όροφο στον οποίο έχει σταθμεύσει (**floor**)

Η κλάση “φορτηγό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, δεν έχει σταθμεύσει σε κάποιον όροφο.

- Το φορτηγό εισέρχεται σε μια ουρά εισόδου γκαράζ, εισάγοντας στην ουρά εισόδου το φορτηγό (**enter**).
- Το φορτηγό σταθμεύει σε έναν όροφο, εάν επιτρέπεται από τον αντίστοιχο μετρητή, αναθέτοντας τον όροφο αυτόν σαν τιμή του ορόφου του και αυξάνοντας τον μετρητή φορτηγών (**park**)
- Το φορτηγό βγαίνει, αναθέτοντας στον όροφο του μηδενική τιμή και μειώνοντας τον μετρητή φορτηγών του ορόφου στον οποίο είχε σταθμεύσει (**exit**).
- Το φορτηγό επιστρέφει τον όροφο στον οποίο έχει σταθμεύσει (**get_floor**).
- Εκτυπώνεται μήνυμα πληρότητας φορτηγών "Truck places are full" (**print_wait_message**).

Η κλάση "γκαράζ":

- έχει τρεις ορόφους (**floor1, floor2, floor3**)
- έχει ένα μέγιστο πλήθος αυτοκινήτων (χωρητικότητα αυτοκινήτων) που επιτρέπεται να είναι σταθμευμένα στο γκαράζ (**car_max**)
- έχει ένα μέγιστο πλήθος φορτηγών (χωρητικότητα φορτηγών) που επιτρέπεται να είναι σταθμευμένα στο γκαράζ (**truck_max**)

Η κλάση "γκαράζ" χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά ανατίθενται οι χωρητικότητες αυτοκινήτων και φορτηγών.
- Ένα αυτοκίνητο ή φορτηγό μπαίνει σε αυτό, μπαίνοντας στον πρώτο όροφο που χωράει. Αν δεν υπάρχει τέτοιος, εκτυπώνεται το σχετικό μήνυμα πληρότητας (**enter**).
- Ένα αυτοκίνητο ή φορτηγό βγαίνει από αυτό, βγαίνοντας από τον όροφο που είναι σταθμευμένο (**exit**).

Η κλάση "όροφος":

- έχει ένα μετρητή πλήθους αυτοκινήτων που είναι σταθμευμένα στον όροφο (**car_counter**)
- έχει ένα μετρητή πλήθους φορτηγών που είναι σταθμευμένα στον όροφο (**truck_counter**)
- έχει ένα σύνολο αυτοκινήτων ή φορτηγών που είναι σταθμευμένα σε αυτόν πλήθους το πολύ όσο το άθροισμα των χωρητικότητων του (**parked_Car_Trucks**)

Η κλάση "όροφος" χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά είναι άδειος.
- Ένα αυτοκίνητο ή φορτηγό μπαίνει σε αυτόν, αν μπορεί να σταθμεύσει, και προστίθεται στο σύνολο των σταθμευμένων οχημάτων (**enter**).
- Αφαιρείται το τελευταίο σταθμευμένο αυτοκίνητο ή φορτηγό από τον όροφο, μηδενίζοντας τη θέση του και βγάζοντάς το από τον όροφο (**exit**).

Η κλάση "ουράΕισόδου":

- έχει ένα γκαράζ στο οποίο οδηγεί (**garage**)
- έχει μια ουρά οχημάτων (**queue_Car_Trucks**)

Η κλάση "ουράΕισόδου" χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά της ανατίθεται το γκαράζ.
- Ένα αυτοκίνητο ή φορτηγό μπαίνει σε αυτή, προστιθέμενο στο τέλος της ουράς της (**enter**).
- Το πρώτο αυτοκίνητο ή φορτηγό βγαίνει από αυτή, αφαιρούμενο από την ουρά της και μπαίνοντας στο γκαράζ (**exit**).

Υλοποιήστε τις κατάλληλες κλάσεις, ορίζοντας μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Θεωρήστε δεδομένο έναν τύπο ουράς με τις λειτουργίες που χρειάζεστε. Μην υλοποιήσετε τη συνάρτηση **main** της προσομοίωσης.

**ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ**

Εξετάσεις 31 Ιανουαρίου 2017

1. (α') Δώστε έναν (τουλάχιστον) λόγο για τον οποίο η γλώσσα Java μπορεί να θεωρηθεί περισσότερο "αντικειμενοστραφής" από την γλώσσα C++.
- (β') Δίδεται το παρακάτω πρόγραμμα Java:

```
public class MainClass {
    public static void main(String[] args) {
        A a1 = new A(); A a2 = new A(10);
        B b1 = new B(); B b2 = new B(100);
        System.out.println("////////////////////////////////////////");
        a1.print(); a2.print(); b1.print(); b2.print();
        System.out.println("////////////////////////////////////////");
        a1 = b1; a1.print(); b1 = b2; a1.print(); } }

class A{
    private int data;
    { System.out.println("A new A was just created"); }
    public A() {
        System.out.println("The A data is " + data); }
    public A(int d) { data = d;
        System.out.println("The A data is " + data); }
    public void print() { System.out.println("The A data is " + data); } }

class B extends A{
    private int data;
    { System.out.println("A new B was just created"); }
    public B() { System.out.println("The B data is " + data); }
    public B(int d) { data = d;
        System.out.println("The B data is " + data); }
    public void print() { super.print();
        System.out.println("The B data is " + data); } }
```

Επίσης, δίδεται και το παρακάτω πρόγραμμα C++:

```
#include <iostream>
using namespace std;

class A{ int data;
public:
    A(int d = 0) {
        cout << "A new A was just created" << endl;
        data = d;
        cout << "The A data is " << data << endl ;}
    void print() { cout << "The A data is " << data << endl ; } };

class B : public A{ int data;
public:
    B(int d = 0) {
        cout << "A new B was just created" << endl;
        data = d;
```

```

        cout << "The B data is " << data << endl ; }
void print() {
    A::print();
    cout << "The B data is " << data << endl; } };

int main() {
    A a1; A a2(10); B b1; B b2(100);
    cout << "/////////////////////////////////" << endl;
    a1.print(); a2.print(); b1.print(); b2.print();
    cout << "/////////////////////////////////" << endl;
    a1 = b1; a1.print(); b1 = b2; a1.print(); }

```

Τα προγράμματα δείχνουν παρόμοια, όμως, δεν έχουν τα ίδια αποτελέσματα. Αιτιολογώντας την απάντησή σας, δώστε τα αποτελέσματα του καθενός. Επίσης, εξηγήσετε συνοπτικά πού και γιατί τα αποτελέσματα διαφέρουν.

2. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++:

```

#include<iostream>
using namespace std;

class A{ int data;
public:
    A(int i=10):data(i)
    { cout << "An A was just constructed with " << i << endl; }
    ~A(){ cout << "An A will be destroyed with " << data << endl; }
    void inc() { data++; }
    int get() { return data; }
    void print() { cout << "The A data is: " << data << endl; } };

class B1 { A data;
public:
    B1(const A& a): data(a)
    { cout << "A B1 was just constructed " << endl; }
    ~B1() { cout << "A B1 will be destroyed with " << data.get() << endl; }
    B1 process() { data.inc(); return *this; }
    void print() { cout << "The B1 data is: " << data.get() << endl ; } };

class B2 { A& data;
public:
    B2(A& a): data(a)
    { cout << "A B2 was just constructed " << endl; }
    ~B2() { cout << "A B2 will be destroyed with " << data.get() << endl; }
    B2& operator=(const B2& b2)
    { cout<< "I am performing a B2 assignment" << endl;
      data = b2.data; return *this;}
    B2 process() { data.inc(); return *this; }
    void print() { cout << "The B2 data is: " << data.get() << endl ; } };

```



```

class B3 { A* data;
public:
    B3(const A& a){
        data = new A(a);
        cout << "A B3 was just constructed " << endl;}
    B3(const B3& b3) {
        data = new A(*(b3.data));
        cout << "A B3 was just constructed by copying " << endl; }
    ~B3() { cout << "A B3 will be destroyed " << endl;
        delete data; }
    B3& operator=(const B3& b3)
        { cout<< "I am performing a B3 assignment" << endl;
          delete data; data = new A(*(b3.data)); return *this; }
    B3 process() { data->inc(); return *this; }
    void print() { cout << "The B3 data is: " << data->get() << endl ; } };

class C { B1 data1; B2 data2; B3 data3;
public:
    C(const B1& b1, const B2& b2, const B3& b3) :
        data1(b1), data2(b2), data3(b3)
        { cout << "A C was just constructed " << endl; }
    ~C() { cout << "A C will be destroyed! " << endl; }
    B1 process1() { return data1.process();}
    B2 process2() { return data2.process();}
    B3 process3() { return data3.process();} };

int main()
{ A a; B1 b1(a); B2 b2(a); B3 b3(a); C c(b1,b2,b3);
  a.print(); b1.print(); b2.print(); b3.print();
  b1 = c.process1(); b2 = c.process2();
  c.process2(); b3 = c.process3();
  cout << "/////////////////////" << endl;
  a.print(); b1.print(); b2.print(); b3.print(); }

```

3. Προκειμένου να ολοκληρώσετε τις σπουδές σας, ενδέχεται να εκπονήσετε Πτυχιακή Εργασία. Η εργασία αυτή θα εντάσσεται σε μια επιστημονική περιοχή των σπουδών σας. Κατά την εκπόνηση της εργασίας, θα πρέπει να μελετήσετε και τη σχετική βιβλιογραφία. Όταν ολοκληρωθεί η εργασία σας, θα πρέπει να συντάξετε και να παραδώσετε μια αναφορά (κείμενο) στην οποία να την περιγράφετε. Στην αναφορά αυτή θα πρέπει να συμπεριλάβετε και την βιβλιογραφία (“references”) την οποία μελετήσατε με τη μορφή συνόλου “βιβλιογραφικών αναφορών”.

Μια βιβλιογραφική αναφορά (“reference”) μπορεί να είναι είτε κάποιο άρθρο από επιστημονικό περιοδικό (“article”) είτε εργασία σε πρακτικά συνεδρίου (“inproceedings”) είτε κεφάλαιο από βιβλίο (“inbook”) είτε βιβλίο (“book”) είτε διατριβή (“thesis”) είτε σελίδα του παγκόσμιου ιστού (“url”).

Μια βιβλιογραφική αναφορά χαρακτηρίζεται από τρεις λέξεις-κλειδιά που αποτελούν χαρακτηριστικά της περιοχής που εντάσσεται (π.χ. προγραμματισμός, λειτουργικά συστήματα, αλγόριθμοι).

Ένα άρθρο από επιστημονικό περιοδικό συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο του άρθρου, τον αριθμό του τόμου του περιοδικού, έναν επιπλέον αριθμό που αφορά υποδιαίρεση του τόμου, τον εκδότη του περιοδικού (π.χ. Elsevier) και χρονολογία έκδοσης του άρθρου.

Μια εργασία σε πρακτικά συνεδρίου συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο της εργασίας, τον τίτλο του συνεδρίου, τον εκδότη των πρακτικών και χρονολογία διεξαγωγής του συνεδρίου.

Ένα κεφάλαιο από βιβλίο συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο του κεφαλαίου, τον τίτλο του βιβλίου, τον εκδότη του βιβλίου και τη χρονολογία έκδοσης του βιβλίου.

Ένα βιβλίο συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο του βιβλίου, τον εκδότη του βιβλίου και τη χρονολογία έκδοσης του βιβλίου.

Μια διατριβή συμπεριλαμβάνει το όνομα του συγγραφέα, τον τίτλο της, το όνομα του ιδρύματος στο οποίο εκπονήθηκε (π.χ. Ε.Κ.Π.Α.) και τη χρονολογία της.

Μια σελίδα παγκόσμιου ιστού συμπεριλαμβάνει τη διεύθυνσή της και την ημερομηνία της τελευταίας επίσκεψής σας σε αυτήν.

Κάθε μία από τις παραπάνω αναφορές αρχικοποιείται με τα στοιχεία που συμπεριλαμβάνει. Δίδεται η δυνατότητα συνολικής εκτύπωσης των στοιχείων της (print) καθώς και η δυνατότητα ανάκτησης των λέξεων-κλειδιών που τη χαρακτηρίζει (get_keywords).

Για στατιστικούς λόγους, μας ενδιαφέρει να μπορούμε να μάθουμε, πόσες αναφορές από το κάθε είδος έχει η βιβλιογραφία μας (get_no_article), (get_no_inproceedings), (get_no_inbook), (get_no_book), (get_no_thesis) και (get_no_url).

Η βιβλιογραφία μιας εργασίας αρχικά είναι κενή. Δίδεται η δυνατότητα προσθήκης μιας αναφοράς στη βιβλιογραφία (add_reference). Στην περίπτωση αυτή, προστίθεται μια ήδη υπάρχουσα αναφορά στη βιβλιογραφία. Μπορεί να γίνει εκτύπωση όλων των αναφορών μιας βιβλιογραφίας (print). Επίσης, δίδεται η δυνατότητα εκτύπωσης των στατιστικών της (print_stats). Τέλος, δίδεται η δυνατότητα εξαγωγής των τριών κυριαρχουσών λέξεων-κλειδιών των αναφορών της βιβλιογραφίας (get_keywords).

Υλοποιήστε την παραπάνω πληροφορία σε C++, υλοποιώντας κατάλληλες κλάσεις. (Σημείωση: Δεν χρειάζεται υλοποίηση συνάρτησης main).

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Εξετάσεις 26 Ιανουαρίου 2018

1. (α') Τι εννοούμε με τον όρο “εμβέλεια” (scope) στον προγραμματισμό; Γιατί είναι κρίσιμη για τον αντικειμενοστραφή προγραμματισμό;
- (β') Δίδεται το παρακάτω πρόγραμμα Java:

```
public class MainClass {
    public static void main(String[] args) { A a;
        a = new C(); a.print(); a = new B(); a.print();
        a = new A(); a.print(); } }

class A{ private int data1;
    {System.out.println("The initial value of A data is: " + data1);}
    A() {this(5);}
    A(int i) { data1 = i;
        System.out.println("The given value of A data is: " + data1);}
    void print() {System.out.println("The current value of A data is: " + data1);} }

class B extends A{
    B() { super(10);}
    B(int i ){super(i);} }

class C extends B{ private int data2;
    C(){this(100);}
    C(int i){super(i); data2 = i;
        System.out.println("The given value of C data is: " + data2);}
    void print() {super.print();
        System.out.println("The current value of C data is: " + data2);} }
```

Επίσης, δίδεται και το παρακάτω πρόγραμμα C++:

```
#include<iostream>
using namespace std;

class A{ int data1;
public:
    A():data1(5) {cout << "The given value of A data is: " << data1 << endl;}
    A(int i):data1(i) { cout << "The given value of A data is: " << data1 << endl;}
    void print() {cout << "The current value of A data is: " << data1 << endl;} };

class B : public A{
public:
    B():A(10) {}
    B(int i ):A(i){} };

class C : public B{ int data2;
public:
    C():data2(100),B(100){cout << "The given value of C data is: " << data2 << endl;}
    C(int i):data2(i),B(i){cout << "The given value of C data is: " << data2 << endl;}
    void print() {B::print();
        cout << "The current value of C data is: " << data2 << endl;} };
```

```
int main(){ A* pa;
    pa = new C(); pa->print(); pa = new B(); pa->print();
    pa = new A(); pa->print(); }
```

Δώστε τα αποτελέσματα των δύο προγραμμάτων, αιτιολογώντας την απάντησή σας. Τα αποτελέσματα των δύο προγραμμάτων διαφέρουν. Αιτιολογήστε γιατί. Τροποποιήστε το πρόγραμμα C++ ώστε να δίνει το ίδιο αποτέλεσμα με το Java πρόγραμμα.

2. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++:

```
#include<iostream>
using namespace std;

class A{ int data;
public:
    A(int i=10):data(i) {cout << "An A just created" << endl;}
    A(const A& a):data(a.data) {cout << "An A just created with CP" << endl;}
    ~A(){cout << "An A to be destroyed with data" << data << endl;}
    void increase(int i) {data*=i;} };

class A1{A data1; A& data2;
public:
    A1(A& a):data1(a),data2(a)
        {cout << "An A1 just created" << endl;}
    ~A1(){cout << "An A1 to be destroyed" << endl;}
    void process() {data1.increase(2); data2.increase(2);} };

class A2{A data1; A* data2;
public:
    A2(const A& a):data1(a) {data2 = new A(a);
        cout << "An A2 just created" << endl;}
    A2(const A2& a2):data1(a2.data1) {data2 = new A(*(a2.data2));
        cout << "An A2 just created with CP" << endl;}
    ~A2(){cout << "An A2 to be destroyed" << endl; delete data2;}
    void process() {data1.increase(10); data2->increase(10);} };

class B{A1& data1; A2 data2;
public:
    B(A1& a1, A2& a2): data1(a1),data2(a2)
        {cout << "A B just created" << endl;}
    ~B(){cout << "A B to be destroyed" << endl;}
    void process(){data1.process(); data2.process();} };

int main(){ A a; A1 a1(a); A2 a2(a);
    B b1(a1,a2); B b2(b1);
    b1.process(); b2.process(); a1.process(); a2.process(); }
```

3. Υλοποιήστε σε C++ την πληροφορία μιας απλουστευμένης βάσης φιλμογραφίας (examK10mdb). Η βάση περιέχει πληροφορίες για (κινηματογραφικές) ταινίες (films), ηθοποιούς (actors) και σκηνοθέτες (directors).

Οι χρήστες μπορούν να ψηφίζουν τους αγαπημένους τους ηθοποιούς και τις αγαπημένες τους ταινίες. Ο ηθοποιός που θα συγκεντρώσει τις περισσότερες ψήφους χαρακτηρίζεται ως ο *πλέον δημοφιλής ηθοποιός*. Ο σκηνοθέτης ο οποίος έχει σκηνοθετήσει ταινίες που συνολικά έχουν συγκεντρώσει τις περισσότερες ψήφους χαρακτηρίζεται ως ο *πλέον δημοφιλής σκηνοθέτης*.

Τόσο οι ηθοποιοί όσο και οι σκηνοθέτες έχουν ονοματεπώνυμο, ημερομηνία γέννησης κι ένα σύνολο ταινιών στις οποίες έχουν συμμετάσχει. Ένας ηθοποιός έχει καταχωρημένο και το πλήθος ψήφων που έχει λάβει.

Τόσο οι ηθοποιοί όσο και οι σκηνοθέτες αρχικοποιούνται με τα στοιχεία του ονοματεπωνύμου τους και της ημερομηνίας γέννησής τους. Αρχικά, θεωρείται ότι δεν έχουν συμμετάσχει σε καμία ταινία. Οι ηθοποιοί, αρχικά, δεν έχουν καμιά ψήφο.

Οι ηθοποιοί και οι σκηνοθέτες συμμετέχουν σε μια ταινία (*participates*), προσθέτοντας την ταινία αυτή σε αυτές που έχουν συμμετάσχει.

Οι ηθοποιοί λαμβάνουν ψήφο (*get_voted*) αυξάνοντας το πλήθος των ψήφων τους κατά 1. Το πλήθος αυτό των ψήφων τους μπορεί να ανακτηθεί (*get_votes*).

Οι σκηνοθέτες συλλέγουν ψήφους (*get_votes*) αθροίζοντας τις ψήφους όλων των ταινιών τις οποίες έχουν σκηνοθετήσει.

Οι ταινίες έχουν έναν τίτλο, ένα έτος παραγωγής, έναν σκηνοθέτη, ένα σύνολο ηθοποιών που συμμετέχουν και το πλήθος ψήφων που έχουν λάβει.

Αρχικά, στις ταινίες καταχωρούνται ο τίτλος τους, το έτος παραγωγής τους, οι ηθοποιοί τους και ο σκηνοθέτης τους. Αρχικά, οι ψήφοι των ταινιών είναι μηδέν.

Μια ταινία λαμβάνει ψήφους (*get_voted*) προσθέτοντας μια μονάδα στις ψήφους της. Το πλήθος αυτό των ψήφων τους μπορεί να ανακτηθεί (*get_votes*).

Τόσο οι ηθοποιοί και οι σκηνοθέτες όσο και οι ταινίες παρέχουν τη δυνατότητα εκτύπωσης των στοιχείων τους (*print*).

Η βάση της φιλμογραφίας περιέχει ηθοποιούς, σκηνοθέτες και ταινίες. Αρχικοποιείται με κάποιους ηθοποιούς, σκηνοθέτες και ταινίες.

Στη βάση, μπορούμε:

- Να ψηφίσουμε έναν ηθοποιό, ψηφίζοντας τον ηθοποιό (*vote*)
- Να ψηφίσουμε μια ταινία, ψηφίζοντας την ταινία (*vote*)
- Να ανακτήσουμε τον “πλέον δημοφιλή ηθοποιό” (*get_favorite_actor*)
- Να ανακτήσουμε τον “πλέον δημοφιλή σκηνοθέτη” (*get_favorite_director*)

Υλοποιήστε την παραπάνω πληροφορία σε C++, υλοποιώντας κατάλληλες κλάσεις, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Θεωρήστε δεδομένο έναν τύπο λίστας, δίδοντας, όμως, τις προδιαγραφές του. (*Σημείωση: Δεν χρειάζεται υλοποίηση συνάρτησης main*).

**ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ**

Εξετάσεις 7 Φεβρουαρίου 2019

1. (α') Δώστε δύο λόγους για τους οποίους η γλώσσα Java μπορεί να θεωρηθεί πιο "Αντικειμενοστραφής" γλώσσα από ότι η C++.

(β') Δίδεται το παρακάτω πρόγραμμα Java:

```
public class Main{
    public static void main(String[] args){
        System.out.println("Initial A count: ");
        A.print_count();
        System.out.println("Initial B count: " );
        B.print_count();

        A a1 = new A();
        a1.print(); a1.print_count(); a1.scope();
        B b1 = new B();
        b1.print(); b1.print_count(); b1.scope();
        a1 = b1;
        a1.print(); a1.print_count(); a1.scope(); } }

class A{
    private int data;
    private static int count;

    static {System.out.println("Initializing A ");}

    public A(){ this(10); }
    public A(int i) { data = i;
        System.out.println("An A just constructed with data: " + data); }
    public void print() {
        System.out.println("Printing message for A data: " + data);
        count++; }
    public static void print_count() { System.out.println(count); }
    public void scope() { System.out.println("Being in A's scope!"); } }

class B extends A{
    private int data;
    private static int count;

    static {System.out. println("Initializing B ");}

    public B() { this(100,200); }
    public B(int i, int j) { super(i);
        data = j;
        System.out.println("A B just constructed with data: " + data); }
    public void print() { super.print();
        System.out.println("Printing message for B data: " + data);
        count++; }
    public static void print_count() { System.out.println(count); }
    public void scope() { System.out.println("Being in B's scope!"); } }
```

Επίσης, δίδεται και το παρακάτω πρόγραμμα C++:

```
#include<iostream>
using namespace std;

class A{
    int data;
    static int count;
public:
    A(int i = 10): data(i) { cout << "An A just constructed with data: "
                           << data << endl; }
    void print() { cout << "Printing message for A data: "
                     << data << endl;
                  count++; }
    static void print_count() { cout << count << endl; }
    void scope() { cout<< "Being in A's scope!" << endl; } };

class B : public A{
    int data;
    static int count;
public:
    B(int i = 100, int j = 200 ): A(i), data(j)
        { cout << "A B just constructed with data: "
              << data << endl; }
    void print() { A::print();
                  cout << "Printing message for B data: "
                       << data << endl;
                  count++; }
    static void print_count() { cout << count << endl; }
    void scope() { cout<< "Being in B's scope!" << endl; } };

int A::count=0;
int B::count=0;

int main(){
    cout << "Initial A count: " << endl;
    A::print_count();
    cout << "Initial B count: " << endl;
    B::print_count();

    A* a1 = new A();
    a1->print(); a1->print_count(); a1->scope();

    B* b1 = new B();
    b1->print(); b1->print_count(); b1->scope();

    a1 = b1;
    a1->print(); a1->print_count(); a1->scope(); }
```

Δώστε τα αποτελέσματα των δύο προγραμμάτων, αιτιολογώντας την απάντησή σας. Τα αποτελέσματα των δύο προγραμμάτων διαφέρουν. Αιτιολογήστε γιατί.

2. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++. Στη συνέχεια, αποσχολιάστε το σχολιασμένο κομμάτι κώδικα. Δώστε πάλι το αποτέλεσμα της εκτέλεσης, αιτιολογώντας την απάντησή σας. Κοινές εκτυπώσεις ή/και αιτιολογήσεις δεν χρειάζεται να επαναληφθούν.

```
#include<iostream>
using namespace std;

class A{
    int data1;
    int& data2;
public:
    A(int d1, int& d2):data1(d1), data2(d2)
        {cout << "An A just constructed" << endl;}
    ~A(){cout << "An A to be destructed with: "
        << data1 << " and " << data2 << endl;}
    A& operator=(const A& a)
        { cout<< "I am performing an A assignment" << endl;
          data1 = a.data1; data2 = a.data2;
          return *this;}
    void inc() { data1++; data2++; }
    void print() {cout << data1 << "    " << data2 << endl;} };

class B{
    int data1;
    int* data2;
    A& data3;
public:
    B(int d1, int* pd2, A& a):data1(d1), data2(pd2), data3(a)
        {cout << "A B just constructed" << endl;}
    ~B(){cout << "A B to be destructed with: "
        << data1 << " and " << *data2 << " and " << endl;
        data3.print(); }
    B& operator=(const B& b)
        { cout<< "I am performing a B assignment" << endl;
          data1 = b.data1; data2 = b.data2; data3 = b.data3;
          return *this;}
    void inc() { data1++; ++(*data2); data3.inc(); } };

int main(){
    int i1 = 10; int i2 = 20;
    A a1(i1, i1); A a2(i2, i2);
    B b1(i1, &i2, a1); B b2(i2, &i1, a2); B b3(b1);

    a1.inc(); a2.inc();
    b1.inc(); b2.inc(); b3.inc();
    cout<< "i1=" << i1 << "    " << "i2=" << i2 << endl;
    /*
    b2 = b3; b3.inc();
    cout<< "i1=" << i1 << "    " << "i2=" << i2 << endl;
    */
}
```


3. Υλοποιήστε σε C++ την πληροφορία ενός ταξιδιωτικού πρακτορείου (TravelAgency). Το ταξιδιωτικό πρακτορείο έχει πελάτες (Customer) και προσφέρει τουριστικά πακέτα (Package_tour). Το τουριστικό πακέτο χαρακτηρίζεται από μια σειρά πόλεων (City), συνοδευόμενες από τον αριθμό των διανυκτερεύσεων σε κάθε μία από αυτές, και τη σειρά από τις συνδέσεις μετακίνησης μεταξύ πόλεων (Travel). Το πρακτορείο δίδει τιμές στα πακέτα. Η τιμή ενός πακέτου προκύπτει από το κόστος του πακέτου επαυξημένη με ένα ποσοστό που αναλογεί στο κέρδος του πρακτορείου. Το κόστος του πακέτου προκύπτει από το κόστος των συνδέσεων μετακίνησης από πόλη σε πόλη επαυξημένο με το συνολικό κόστος της διαμονής στις πόλεις. Ανά πάσα στιγμή, το πρακτορείο μπορεί να υπολογίσει το συνολικό κέρδος του από τα πακέτα που έχει πουλήσει.

Ένας πελάτης έχει μια ταυτότητα και χαρακτηρίζεται από το σύνολο των πακέτων που έχει αγοράσει. Αρχικά έχει μόνο ταυτότητα και δεν έχει αγοράσει κανένα πακέτο. Αγοράζει ένα πακέτο (`buy_package`), προσθέτοντας το πακέτο στα πακέτα που έχει αγοράσει. Δίνεται η δυνατότητα να εκτυπωθούν τα πακέτα που έχει αγοράσει (`print`).

Μια πόλη έχει ένα όνομα κι ένα κόστος διανυκτέρευσης και αρχικοποιείται με όλη την πληροφορία αυτή. Μπορούμε να ανακτήσουμε το όνομά της (`get_name`). Επίσης, μπορούμε να ανακτήσουμε το κόστος διανυκτέρευσης (`get_cost`).

Μια σύνδεση μετακίνησης μεταξύ πόλεων χαρακτηρίζεται από τις δύο πόλεις που συνδέει και το κόστος της. Η σύνδεση αυτή μπορεί να είναι είτε αεροπορική πτήση είτε ακτοπλοϊκή σύνδεση είτε δρομολόγιο τρένου είτε δρομολόγιο λεωφορείου. Στην περίπτωση της αεροπορικής πτήσης συμπεριλαμβάνεται κι η αεροπορική εταιρεία. Στην περίπτωση της ακτοπλοϊκής σύνδεσης, συμπεριλαμβάνεται και το όνομα του πλοίου. Όλες οι περιπτώσεις σύνδεσης μετακίνησης αρχικοποιούνται με την πληροφορία που τις χαρακτηρίζει. Πρέπει να παρέχεται η δυνατότητα να ανακτήσουμε το κόστος της (`get_cost`). Επίσης, πρέπει να δίνεται η δυνατότητα εκτύπωσης όλων των στοιχείων της (`print`). Κατά την εκτύπωση, εκτυπώνεται και το μέσο μετακίνησης.

Ένα πακέτο χαρακτηρίζεται από τη σειρά των πόλεων που το απαρτίζουν όπου η κάθε πόλη συνοδεύεται από τον αριθμό των διανυκτερεύσεων σε αυτές —η πρώτη πόλη και η τελευταία είναι η πόλη αναχώρησης και η πόλη άφιξης, αντίστοιχα, οπότε δεν υπάρχει διανυκτέρευση σε αυτές—. Το πακέτο χαρακτηρίζεται επίσης, κι από τη σειρά των συνδέσεων μετακίνησης ανάμεσα στις πόλεις του. Ένα πακέτο αρχικοποιείται με όλη την παραπάνω πληροφορία. Για ένα πακέτο, μπορούμε να υπολογίσουμε το κόστος του (`get_cost`) αθροίζοντας τα κόστη διανυκτερεύσεων στις πόλεις που το αποτελούν καθώς και τα κόστη των μετακινήσεων μεταξύ των πόλεών του. Επίσης, μπορούμε να το εκτυπώσουμε (`print`), εκτυπώνοντας τα ονόματα των πόλεων που το απαρτίζουν, τον αριθμό διανυκτερεύσεων σε κάθε μία από αυτές και κάνοντας εκτύπωση και των συνδέσεων μετακίνησής του.

Το τουριστικό πρακτορείο, αρχικά, δεν έχει κανένα πελάτη και κανένα τουριστικό πακέτο. Μπορεί να προστεθεί πελάτης (`add_customer`), προσθέτοντας τον στους ήδη υπάρχοντες. Μπορεί να προστεθεί τουριστικό πακέτο (`add_package_tour`), δημιουργώντας το και προσθέτοντάς το στα ήδη υπάρχοντα. Ένας ήδη υπάρχων πελάτης μπορεί να αγοράσει ένα πακέτο (`buy_package`). Αν δεν είναι στους ήδη υπάρχοντες, πρέπει και να προστεθεί σε αυτούς. Για ένα πακέτο, καθορίζουμε την τιμή του (`define_price`) αυξάνοντας το κόστος του κατά ένα ποσοστό κέρδους το οποίο αποτελεί σταθερά του πρακτορείου. Επίσης, μπορεί να υπολογιστεί το κέρδος που έχει το πρακτορείο με βάση τα πακέτα που έχουν αγοράσει οι πελάτες του (`evaluate_profit`). Τέλος, για έναν πελάτη, μπορεί να εκτυπώσει τα πακέτα που έχει αγοράσει (`print`).

Υλοποιήστε την παραπάνω πληροφορία σε C++, υλοποιώντας κατάλληλες κλάσεις, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Θεωρήστε δεδομένους τους τύπους δεδομένων που θα χρησιμοποιήσετε, δίδοντας, όμως, τις προδιαγραφές τους. (Σημείωση: Δεν χρειάζεται υλοποίηση συνάρτησης `main`).

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Εξετάσεις 28 Ιανουαρίου 2020

1. (α') Στον αντικειμενοστραφή προγραμματισμό συναντάμε τις σχέσεις *"is a"* και *"has a"* μεταξύ κλάσεων. Τι εννοούμε με τις σχέσεις αυτές; Δώστε παράδειγμα κλάσεων σε C++ που να σχετίζονται μεταξύ τους με αυτές τις σχέσεις.

- (β') Δίδεται το παρακάτω πρόγραμμα C++:

```
#include <iostream>
using namespace std;
```

```
class Data { int data;
public:
    Data(int i = 0) : data(i) { cout << "Data constructed " << endl; }
    ~Data() { cout << "Data to be destructed with: " << data << endl; }
    void inc() { data++; }
    void print() { cout << "Printing Data with: " << data << endl; } };

class A{ Data* data;
public:
    A(const Data& dt) { cout << "A constructed " << endl; data = new Data(dt); }
    A(const A& a) { cout << "A constructed by CP " << endl;
                    data = new Data(*(a.data)); }
    virtual ~A() { cout << "A to be destructed with Data: " << endl; data->print();
                  delete data; }
    void inc() { data->inc(); }
    virtual void print() { cout << "Printing a A with: " << endl; data->print();} };
```

```
class B : public A{ Data& data;
public:
    B(Data& dt) : data(dt), A(dt) { cout << "B constructed " << endl; }
    ~B() { cout << "B to be destructed with Data: " << endl; data.print(); }
    void inc() { A::inc(); data.inc(); }
    void print() { A::print(); cout << "Printing a B with: " << endl; data.print();} };
```

```
class C : public B{ Data data;
public:
    C(Data& dt) : data(dt), B(dt) { cout << "C constructed " << endl; }
    ~C() { cout << "C to be destructed with Data: " << endl; data.print(); }
    void inc() { B::inc(); data.inc(); }
    void print() { B::print(); cout << "Printing a C with: " << endl; data.print();} };
```

```
class Fat1{ A data1; A& data2;
public:
    Fat1(A& a): data1(a), data2(a) { cout << " Fat1 just constructed " << endl; }
    ~Fat1() { cout << "A Fat1 to be destructed " << endl; }
    void inc() { data1.inc(); data2.inc(); }
    void print() { data1.print(); data2.print(); } };
```

```
class Fat2{ B data1; B& data2;
public:
    Fat2(B& b): data1(b), data2(b) { cout << " Fat2 just constructed " << endl; }
```

```

~Fat2() { cout << "A Fat2 to be destructed " << endl; }
void inc() { data1.inc(); data2.inc(); }
void print() { data1.print(); data2.print(); } };

int main(){ Data dt(200); A a(dt); B b(dt); C c(dt);
a.inc(); b.inc(); c.inc(); a.print(); b.print(); c.print();

Fat1 f1(a); Fat2 f2(b);
f1.inc(); f2.inc(); f1.print(); f2.print();

A* p1 = &a; A* p2 = &b; A* p3 = &c;
p1->inc(); p2->inc(); p3->inc(); p1->print(); p2->print(); p3->print(); }

```

Δώστε τα αποτελέσματα της εκτέλεσης του προγράμματος, αιτιολογώντας την απάντησή σας.

2. Ας θεωρήσουμε έναν φοιτητή που παρακολουθεί ένα πρόγραμμα προπτυχιακών σπουδών ενός πανεπιστημιακού τμήματος. Το πρόγραμμα αυτό αποτελείται από μαθήματα. Τα μαθήματα μπορεί να είναι είτε υποχρεωτικά μαθήματα είτε προαιρετικά μαθήματα του τμήματος. Ο αριθμός των υποχρεωτικών μαθημάτων είναι καθορισμένος εξ αρχής. Επίσης, υπάρχουν και ελεύθερα μαθήματα τα οποία προσφέρονται από άλλα τμήματα. Κάθε μάθημα έχει ένα όνομα καθώς και το πλήθος των ECTS που το χαρακτηρίζει.

Κατηγορία υποχρεωτικών μαθημάτων είναι τα υποχρεωτικά μαθήματα κατεύθυνσης που πρέπει υποχρεωτικά να παρακολουθήσει επιτυχώς ένας φοιτητής στην περίπτωση που αποφασίσει να ακολουθήσει την κατεύθυνση αυτή. Έστω ότι οι κατευθύνσεις έχουν τιμές 'Α' και 'Β'. Ένα υποχρεωτικό μάθημα κατεύθυνσης περιέχει και την κατεύθυνση στην οποία ανήκει. Ο αριθμός των υποχρεωτικών μαθημάτων κατεύθυνσης είναι καθορισμένος εξ αρχής και κοινός και για τις δύο κατευθύνσεις.

Κατηγορία προαιρετικών μαθημάτων είναι τα βασικά μαθήματα ειδίκευσης. Οι ειδικεύσεις έχουν τιμές "E1", "E2", "E3", "E4", "E5", "E6". Ένα βασικό μάθημα ειδίκευσης περιέχει και την ειδίκευση της οποίας είναι βασικό (μόνο μία). Για να γίνει κατοχύρωση ειδίκευσης (**specialization_ok**), ένας φοιτητής πρέπει να παρακολουθήσει επιτυχώς τουλάχιστον έναν συγκεκριμένο αριθμό μαθημάτων από την ειδίκευση αυτή. Ο αριθμός αυτός είναι κοινός για τα μαθήματα όλων των ειδικεύσεων και μπορεί να μεταβάλλεται μέσω μιας διαδικασίας αλλαγής τιμής (**change_min**).

Τα ελεύθερα μαθήματα συνοδεύονται κι από το όνομα του τμήματος που τα προσφέρει.

Κάθε φοιτητής έχει έναν αριθμό μητρώου, έχει δηλώσει κατεύθυνση και έχει παρακολουθήσει επιτυχώς ένα σύνολο μαθημάτων. Για τον φοιτητή, μπορεί να ζητηθεί να εκτυπωθεί η πληροφορία αυτή (**print**). Στην περίπτωση αυτή ζητείται να εκτυπώνεται όλη η πληροφορία των μαθημάτων. Επίσης, μπορεί να ζητηθεί να εκτιμηθεί αν έχει κατοχυρώσει ειδίκευση (**specialization_ok**), αν έχει κατοχυρώσει συγκεκριμένη ειδίκευση (**specialization_ok**) και αν έχει ολοκληρώσει τις υποχρεώσεις λήψης πτυχίου (**finished**). Για το τελευταίο, πρέπει να έχει παρακολουθήσει επιτυχώς όλα τα υποχρεωτικά μαθήματα, όλα τα υποχρεωτικά μαθήματα κατεύθυνσης και να έχει συμπληρώσει τον απαιτούμενο αριθμό ECTS. Επίσης, υπάρχει ένα μέγιστο στο πλήθος των ECTS από ελεύθερα μαθήματα που μπορεί να ληφθεί υπόψη για την ολοκλήρωση των υποχρεώσεων λήψης πτυχίου. Θεωρήστε ότι ο φοιτητής αρχικοποιείται με τα στοιχεία του καθώς και τα μαθήματα που έχει παρακολουθήσει επιτυχώς.

Κάθε μια από τις παραπάνω οντότητες, μπορεί να αρχικοποιηθεί, παρέχοντας τις τιμές των επί μέρους στοιχείων της και να εκτυπωθεί, εκτυπώνοντας όλες τις πληροφορίες που σχετίζονται με αυτήν (**print**).

Υλοποιήστε την παραπάνω πληροφορία σε C++, υλοποιώντας κατάλληλες κλάσεις, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. (Σημείωση: Δεν χρειάζεται υλοποίηση συνάρτησης **main**).

ΠΡΟΣΟΧΗ -προς αποφυγή παρεξηγήσεων: Το ερώτημα είναι εμπνευσμένο από το ΠΠΣ του Τμήματός μας αλλά οι προδιαγραφές του δεν ταυτίζονται με αυτό!

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Εξετάσεις 2 Φεβρουαρίου 2022 (Slot1: 9:00 - 10:30)

1. (α') Τι εννοούμε με τον όρο “προγραμματισμός με στοιχεία” (modular programming) και τι με τον όρο “αφαίρεση δεδομένων” (data abstraction);
- (β') Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++:

```
#include<iostream>

using namespace std;

class A{
    int data;
public:
    A():data(10) { cout << "Constructing an A " << endl; }
    A(const A& a):data(a.data) { cout << "Constructing an A by CP " << endl; }
    ~A() { cout << "Destructing an A with: " << data << endl; }
    void inc() { data++; }
};

class B{
    A data;
public:
    B() { cout << "Constructing a B " << endl; }
    B(const B& b) : data(b.data) { cout << "Constructing a B by CP " << endl; }
    ~B() { cout << "Destructing a B " << endl; }
    void inc() { data.inc(); }
};

class C{
    A* data;
public:
    C() { cout << "Constructing a C " << endl;  data = new A(); }
    C(const C& c) { cout << "Constructing a C by CP " << endl;
                  data = new A(*(c.data)); }
    ~C() { cout << "Destructing a C " << endl; delete data; }
    void inc() { data->inc(); }
};

void bla1(B b) { b.inc(); }
void bla2(C& c) { c.inc(); }

int main(){

    B b; C c;

    bla1(b);
    bla2(c);

    return 0;
}
```

2. Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση εισόδου επισκεπτών σε ένα κτήριο δημόσιας υπηρεσίας. Προκειμένου να γίνει η υλοποίηση αυτή θεωρούμε τις κλάσεις: “επισκέπτης” (Visitor), “κτήριο” (Building), “όροφος” (Floor) και “γραφείο” (Office). Τα γραφεία έχουν μια μέγιστη χωρητικότητα το καθένα. Για ευκολία, θεωρείστε ότι αυτή έχει την ίδια τιμή για όλα τα γραφεία.

Η κλάση “επισκέπτης”:

- έχει ένα κτήριο που θέλει να επισκεφτεί (`building`)
- έχει ένα αριθμό γραφείου, του γραφείου που επιθυμεί να επισκεφτεί (`no_office`). Ο αριθμός γραφείου είναι μια δόμηση πληροφορίας που αποτελείται από δύο ακεραίους: τον όροφο του γραφείου και τον αριθμό στον όροφο αυτό.

Η κλάση “επισκέπτης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά του ανατίθεται το κτήριο και αρχικοποιείται ο αριθμός γραφείου του με τα στοιχεία του αριθμού του γραφείου που θέλει να επισκεφτεί.
- Επιστρέφεται ο όροφος που αντιστοιχεί στον αριθμό γραφείου του (`get_floor`).
- Επιστρέφεται ο αριθμός που αντιστοιχεί στον αριθμό γραφείου του (`get_room`).
- Μπαίνει στο κτήριο, αν του επιτρέπεται (`enter`).

Η κλάση “κτήριο”:

- έχει τέσσερεις ορόφους (`floors`)

Η κλάση “κτήριο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά δεν γίνεται κάποια επιπλέον ενέργεια πέρα από τις αυτόματες αρχικοποιήσεις -οπότε λαμβάνεται πρόνοια ώστε αυτές να έχουν τα απαραίτητα δεδομένα.
- Ένας επισκέπτης μπαίνει στο κτήριο, μπαίνοντας στον όροφο που βρίσκεται το γραφείο που θέλει να επισκεφτεί, αν αυτό επιτρέπεται (`enter`).

Η κλάση “όροφος”:

- έχει δέκα γραφεία (`offices`)

Η κλάση “όροφος” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά δεν γίνεται κάποια επιπλέον ενέργεια πέρα από τις αυτόματες αρχικοποιήσεις -οπότε λαμβάνεται πρόνοια ώστε αυτές να έχουν τα απαραίτητα δεδομένα.
- Ένας επισκέπτης μπαίνει στον όροφο, μπαίνοντας στο γραφείο που επιθυμεί, αν του επιτρέπεται (`enter`).

Η κλάση “γραφείο”:

- έχει μια μέγιστη χωρητικότητα (`capacity`)
- έχει ένα μετρητή για το πλήθος των επισκεπτών που βρίσκονται μέσα σε αυτό τη στιγμή αυτή (`no_of_visitors`)

Η κλάση “γραφείο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά είναι άδειο και του ανατίθεται η μέγιστη χωρητικότητά του.
- Ένας επισκέπτης μπαίνει στο γραφείο αν δεν υπερβαίνεται η μέγιστη χωρητικότητά του, αυξάνοντας το μετρητή του πλήθους των επισκεπτών του (`enter`).

Υλοποιήστε τα παραπάνω μέσω καταλλήλων κλάσεων, ορίζοντας μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά.

(Σημείωση: Δεν χρειάζεται υλοποίηση συνάρτησης `main`).

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ
Εξετάσεις 2 Φεβρουαρίου 2022 (Slot2: 11:30 - 13:00)

1. (α') Τι επιτυγχάνουμε με το μηχανισμό του εγκλωβισμού (encapsulation) στον αντικειμενοστραφή προγραμματισμό; Που αυτός είναι χρήσιμος στον προγραμματισμό, γενικότερα;
- (β') Δίνεται το παρακάτω πρόγραμμα C++:

```
#include<iostream>
using namespace std;

class A{
    int data;
public:
    A(): data(10) { cout << "Constructing an A" << endl; }
    A(const A& a): data(a.data) { cout << "Constructing an A by CP" << endl;}
    ~A(){ cout << "Destructing an A with data:" << data << endl; }
    void inc() { data++; } };

class B{
    A data;
public:
    B(const A& a):data(a) { cout << "Constructing a B" << endl; }
    ~B(){ cout << "Destructing a B " << endl; }
    virtual void bla1() { cout << "B::bla1() " << endl; data.inc();}
    static void bla2() { cout << "In B but nothing done! " << endl;} };

class C : public B{
    A& data;
public:
    C(A& a):B(a),data(a) { cout << "Constructing a C" << endl;}
    ~C(){ cout << "Destructing a C " << endl; }
    void bla1() { cout << "C::bla1() " << endl; data.inc();}
    static void bla2() { cout << "In C but nothing done! " << endl;} };

void foopB(B* pb) {
    pb->bla1(); pb->bla2(); }

void foopC(C* pc) {
    pc->bla1(); pc->bla2(); }

int main(){
    A a; B b(a); C c(a);
    foopB(&c);
    foopC(&b);
    return 0; }
```

Το πρόγραμμα αυτό δεν μεταγλωττίζεται εξ αιτίας λάθους στο σώμα της συνάρτησης main. Ποιό είναι αυτό και γιατί είναι λάθος; Διαγράψτε την εντολή με το πρόβλημα και, στη συνέχεια, δώστε το αποτέλεσμα της εκτέλεσης του προγράμματος αιτιολογώντας την απάντησή σας.

2. Κατά την εκπόνηση μιας επιστημονικής εργασίας, οφείλουμε να μελετήσουμε την σχετική βιβλιογραφία. Στην αναφορά στην οποία θα περιγράψουμε τα αποτελέσματα της εργασίας μας, οφείλουμε να συμπεριλάβουμε και την βιβλιογραφία (“references”) την οποία μελετήσαμε με τη μορφή συνόλου “βιβλιογραφικών αναφορών”.

Ας υποθέσουμε ότι μια βιβλιογραφική αναφορά (“reference”) μπορεί να είναι είτε κάποιο άρθρο από επιστημονικό περιοδικό (“article”) είτε εργασία σε πρακτικά συνεδρίου (“inproceedings”) είτε βιβλίο (“book”).

Μια βιβλιογραφική αναφορά χαρακτηρίζεται από τρεις λέξεις-κλειδιά που αποτελούν χαρακτηριστικά της περιοχής που εντάσσεται (π.χ. προγραμματισμός, λειτουργικά συστήματα, αλγόριθμοι).

Ένα άρθρο από επιστημονικό περιοδικό συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο του άρθρου, τον αριθμό του τόμου του περιοδικού, έναν επιπλέον αριθμό που αφορά υποδιαίρεση του τόμου, τον εκδότη του περιοδικού (π.χ. Elsevier) και χρονολογία έκδοσης του άρθρου.

Μια εργασία σε πρακτικά συνεδρίου συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο της εργασίας, τον τίτλο του συνεδρίου, τον εκδότη των πρακτικών και χρονολογία διεξαγωγής του συνεδρίου.

Ένα βιβλίο συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο του βιβλίου, τον εκδότη του βιβλίου και τη χρονολογία έκδοσης του βιβλίου.

Κάθε μία από τις παραπάνω αναφορές αρχικοποιείται με τα στοιχεία που συμπεριλαμβάνει. Δίδεται η δυνατότητα συνολικής εκτύπωσης των στοιχείων της (print) καθώς και η δυνατότητα ανάκτησης των λέξεων-κλειδιών που τη χαρακτηρίζει (get_keywords).

Η βιβλιογραφία μιας εργασίας αρχικά είναι κενή. Δίδεται η δυνατότητα προσθήκης μιας αναφοράς στη βιβλιογραφία (add_reference). Στην περίπτωση αυτή, προστίθεται μια ήδη υπάρχουσα αναφορά στη βιβλιογραφία. Μπορεί να γίνει εκτύπωση όλων των αναφορών μιας βιβλιογραφίας (print). Επίσης, για στατιστικούς λόγους, μας ενδιαφέρει να μπορούμε να μάθουμε, πόσες αναφορές από το κάθε είδος έχει η βιβλιογραφία μας — (get_no_article), (get_no_inproceedings), και (get_no_book)— και δίδεται η δυνατότητα εκτύπωσης όλων αυτών των στατιστικών της (print_stats). Τέλος, δίδεται η δυνατότητα εξαγωγής των τριών κυριαρχουσών λέξεων-κλειδιών των αναφορών της βιβλιογραφίας (get_keywords).

Υλοποιήστε την παραπάνω πληροφορία σε C++, υλοποιώντας κατάλληλες κλάσεις. Για την περίπτωση που το χρειάζεστε, θεωρείστε δεδομένο έναν τύπο λίστας και δεν χρειάζεται να τον υλοποιήσετε.

(Σημείωση: Δεν χρειάζεται υλοποίηση συνάρτησης main).

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ
Εξετάσεις 2 Φεβρουαρίου 2022 (Slot3: 14:00 - 15:30)

1. (α') Στις γλώσσες αντικειμενοστραφούς προγραμματισμού, κατά τον ορισμό μιας κλάσης χωρίζουμε το δημόσιο μέρος της από το ιδιωτικό. Ποιός μηχανισμός απαιτείται από τη γλώσσα προκειμένου να επιτύχουμε τη διάκριση αυτή; Ο μηχανισμός αυτός είναι απαιτούμενο μόνο για τις γλώσσες του αντικειμενοστραφούς προγραμματισμού;
- (β') Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του παρακάτω προγράμματος C++:

```
#include<iostream>

using namespace std;

class A{
    int data;
public:
    A():data(10) { cout << "Constructing an A " << endl; }
    ~A(){ cout << "Destructing an A with: " << data << endl; }
    void inc() { cout << "A::inc()" << endl; data++; }
    void bla() { cout << "A::bla()" << endl; inc(); }
};

class B : public A{
    int data;
public:
    B():data(20) { cout << "Constructing a B " << endl; }
    ~B(){ cout << "Destructing a B with: " << data << endl; }
    void inc() { cout << "B::inc()" << endl; data++; }
    virtual void bla() { cout << "B::bla()" << endl; inc(); }
};

class C : public B{
    int data;
public:
    C():data(30) { cout << "Constructing a C " << endl; }
    ~C(){ cout << "Destructing a C with: " << data << endl; }
    void inc() { cout << "C::inc()" << endl; data++; }
    void bla() { cout << "C::bla()" << endl; inc(); }
};

void foo1(A* pa) { pa->bla(); }
void foo2(B* pb) { pb->bla(); }

int main(){
    C c;
    foo1(&c);
    foo2(&c);

    return 0;
}
```


2. Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση της εισόδου αυτοκινήτων σε ένα γκαράζ. Στο γκαράζ θεωρούμε ότι σταθμεύουν τόσο (επιβατικά) αυτοκίνητα όσο και φορτηγά, σε τρεις ορόφους. Το γκαράζ έχει μια μέγιστη χωρητικότητα για στάθμευση επιβατικών αυτοκινήτων και μια μέγιστη χωρητικότητα για στάθμευση φορτηγών, ισοκατανεμημένες στους τρεις ορόφους.

Προκειμένου να γίνει η υλοποίηση αυτή θεωρούμε τις κλάσεις: “επιβατικό αυτοκίνητο” (Car), “φορτηγό” (Truck), “γκαράζ” (Garage) και “όροφος” (Floor).

Η κλάση “επιβατικό αυτοκίνητο”:

- έχει ένα γκαράζ στο οποίο επιθυμεί να σταθμεύσει (garage)
- έχει (μία ένδειξη σε) έναν όροφο στον οποίο έχει σταθμεύσει (floor)

Η κλάση “επιβατικό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, δεν έχει σταθμεύσει σε κάποιον όροφο και του ανατίθεται το γκαράζ στο οποίο επιθυμεί να σταθμεύσει.
- Το αυτοκίνητο εισέρχεται στο γκαράζ εάν αυτό είναι δυνατό (enter).
- Το αυτοκίνητο σταθμεύει σε έναν όροφο, εάν επιτρέπεται, αναθέτοντας τον όροφο αυτόν σαν τιμή του ορόφου του (park).
- Εκτυπώνεται μήνυμα πληρότητας αυτοκινήτων "Full of Cars so I have to wait!" (print_wait_message).

Η κλάση “φορτηγό αυτοκίνητο”:

- έχει ένα γκαράζ στο οποίο επιθυμεί να σταθμεύσει (garage)
- έχει (μία ένδειξη σε) έναν όροφο στον οποίο έχει σταθμεύσει (floor)

Η κλάση “φορτηγό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, δεν έχει σταθμεύσει σε κάποιον όροφο και του ανατίθεται το γκαράζ στο οποίο επιθυμεί να σταθμεύσει.
- Το φορτηγό εισέρχεται στο γκαράζ εάν αυτό είναι δυνατό (enter).
- Το φορτηγό σταθμεύει σε έναν όροφο, εάν επιτρέπεται, αναθέτοντας τον όροφο αυτόν σαν τιμή του ορόφου του (park).
- Εκτυπώνεται μήνυμα πληρότητας φορτηγών "Full of Trucks so I have to wait!" (print_wait_message).

Η κλάση “όροφος”:

- έχει μια μέγιστη χωρητικότητα επιβατικών αυτοκινήτων (car_capacity)
- έχει μια μέγιστη χωρητικότητα φορτηγών (truck_capacity)
- έχει ένα μετρητή πλήθους επιβατικών αυτοκινήτων που είναι σταθμευμένα στον όροφο (car_counter)
- έχει ένα μετρητή πλήθους φορτηγών που είναι σταθμευμένα στον όροφο (truck_counter)

Η κλάση “όροφος” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά είναι άδειος και του ανατίθενται οι μέγιστες χωρητικότητες επιβατικών αυτοκινήτων και φορτηγών.
- Ένα επιβατικό αυτοκίνητο μπαίνει σε αυτόν, αν επιτρέπεται από την αντίστοιχη χωρητικότητα, και αυξάνεται ο μετρητής των σταθμευμένων επιβατικών αυτοκινήτων (enter).
- Ένα φορτηγό μπαίνει σε αυτόν, αν επιτρέπεται από την αντίστοιχη χωρητικότητα, και αυξάνεται ο μετρητής των σταθμευμένων φορτηγών (enter).

Η κλάση “γκαράζ”:

- έχει τρεις ορόφους (floor1, floor2, floor3)

Η κλάση “γκαράζ” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά δεν γίνεται κάποια επιπλέον ενέργεια πέρα από τις αυτόματες αρχικοποιήσεις των ορόφων του στις οποίες διαβιβάζονται οι χωρητικότητες επιβατικών αυτοκινήτων και φορτηγών.
- Ένα επιβατικό αυτοκίνητο ή φορτηγό μπαίνει σε αυτό, αν χωράει, μπαίνοντας στον πρώτο όροφο που χωράει. Αν δεν υπάρχει τέτοιος, εκτυπώνεται το σχετικό μήνυμα πληρότητας (enter).

Υλοποιήστε κατάλληλες κλάσεις, ορίζοντας μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά.

(Σημείωση: Δεν χρειάζεται υλοποίηση συνάρτησης main).

**ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10:ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ**

Εξετάσεις 20 Σεπτεμβρίου 2018

1. (α') Στον αντικειμενοστραφή προγραμματισμό μιλάμε για “αφαίρεση στα δεδομένα” καθώς και για “σύνθεση” και “κληρονομικότητα” μεταξύ κλάσεων. Πού αναφέρεται ο καθένας όρος από αυτούς; Δώστε ένα παράδειγμα όπου η αφαίρεση στα δεδομένα είναι χρήσιμη στην υλοποίησή του αλλά η κληρονομικότητα δεν είναι απαραίτητη.

- (β') Δίδεται το παρακάτω πρόγραμμα C++:

```
#include<iostream>
using namespace std;

class A{ int data;
public:
    A(int i=100) : data(i)
        { cout << "I just constructed an A with: " << data << endl; }
    A(const A& a) : data(a.data)
        { cout << "I just constructed an A by copying with data "
          << data << endl; }
    ~A() { cout << "Destructing an A with data " << data << endl; }
    int get_data() { return data; }
    void change() { data += 100; } };

class FatA{
    A data1; A* data2; A& data3;
public:
    FatA(A& a) : data1(a), data3(a)
        { data2 = new A(a);
          cout << "I just constructed a FatA with: "
                << data1.get_data() << " " << data2->get_data() << " "
                << data3.get_data() << endl; }
    FatA(FatA& fa) : data1(fa.data1), data3(fa.data3)
        { data2 = new A(*(fa.data2)); }
    ~FatA() { cout << "Destructing a FatA " << endl; delete data2; }
    void change() { data1.change(); data2->change(); data3.change();} };

int main(){ A a;
            FatA f1(a); FatA f2 = f1;
            f1.change(); f2.change();
            return 0; }
```

και το παρακάτω πρόγραμμα Java:

```
public class first {
    public static void main(String[] args) {
        A a = new A(100);
        FatA f1 = new FatA(a); FatA f2 = f1;
        f1.change(); f2.change();
        System.out.println("Printing f1 data"); f1.print();
        System.out.println("Printing f2 data"); f2.print(); } }
```

```

class A{
private int data;
A(int i){ data = i;
        System.out.println("I just constructed an A with " + data); }
int get_data(){ return data;}
void change(){ data += 100; } }

class FatA{
    private A data1;
    private A data2;
    private A data3;

    FatA(A a)
    {   data1 = a; data2 = a; data3 = a;
        System.out.println("I just constructed a FatA with: " +
            data1.get_data() + " " + data2.get_data() + " " +
            data3.get_data()); }
    void change() { data1.change(); data2.change(); data3.change(); }
    void print() {System.out.println( " The FatA data are: " +
        data1.get_data() + " " +
        data2.get_data() + " " +
        data3.get_data());} }

```

Δώστε τα αποτελέσματα της εκτέλεσης των παραπάνω προγραμμάτων, αιτιολογώντας τα. Εντοπίστε τις ουσιαστικές διαφορές στα αποτελέσματα αυτά και αιτιολογείστε τις.

- Δίδεται το παρακάτω πρόγραμμα C++. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του, αποσχολιάζοντας μιά γραμμή κάθε φορά στον ορισμό της κλάσης Musical.

```

#include<iostream>
using namespace std;

class Actor{ int strength;
public :
    Actor(int s=10) : strength(s) {cout << "Creating an Actor" << endl;}
    Actor(const Actor& actor) : strength(actor.strength)
        {cout << "Creating a Copy of an Actor" << endl;}
    virtual ~Actor() {cout << "Disappearing ..." << strength << endl;}
    int get_strength() {return strength;}
    void tiring() {if (strength > 0) strength-- ;}
    virtual void act() {cout << "words, words, words ..." << endl; tiring();} };

```

```

class ActorDancer : public Actor{ int strength;
public:
    ActorDancer(int as=5, int s=100) : Actor(as), strength(s)
        {cout << "Creating an Actor who Dances" << endl;}
    virtual ~ActorDancer()
        {if (get_strength()> 80) cout << "Exiting but no problem at all" << endl ;
         else { if (get_strength()> 50) cout << "Exiting but still keeping" << endl ;
               else cout << "Exiting exhausted" << endl;}
         cout << strength << endl;}
    int get_strength() {return Actor::get_strength() + strength;}
    void tiring() {cout << strength << endl;
                  if (strength > 20 ) strength -= 20;
                  cout << strength << endl;}
    virtual void act() {cout << "I am dancing in the streets" << endl; tiring();} };

class ActorSingerDancer : public ActorDancer{
public:
    ActorSingerDancer(int astrength=30, int dstrength=50) :
        ActorDancer(astrength, dstrength)
        {cout << "Creating an Actor who Sings and Dances" << endl;}
    virtual void act() {ActorDancer::act();
                      cout << "I am singing under the stars!" << endl;
                      tiring(); Actor::tiring();} };

class Musical{
    ActorSingerDancer& starring1; ActorSingerDancer& starring2;
    ActorDancer& dancer; Actor& actor;
public :
    Musical(ActorSingerDancer& s1, ActorSingerDancer& s2, ActorDancer& d, Actor& a) :
        starring1(s1), starring2(s2), dancer(d), actor(a)
        {cout << "The show starts!" << endl;
         scene(s1,s2); scene(d,a);}
    ~Musical() {cout << "The End" << endl;}
//    void scene(Actor a1, Actor a2)
//    void scene(Actor& a1, Actor& a2)
        { a1.act(); a2.act();} };

int main(){ ActorSingerDancer ES(60); ActorSingerDancer RG(40,70);
           ActorDancer AD; Actor A;
           Musical la_la_land(ES, RG, AD, A);
           return 0; }

```

3. Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση ενός θερινού σινεμά. Έστω ότι η προσομοίωση αυτή θεωρεί τις εξής κλάσεις:

- κλάση: “Ταμείο” που προσομοιώνει την εξυπηρέτηση στο ταμείο του θερινού
- κλάση: “Bar” που προσομοιώνει την εξυπηρέτηση στο μπαρ του θερινού
- κλάση: “Cinema” που προσομοιώνει την εξυπηρέτηση στο θερινό σινεμά.

Η κλάση “Ταμείο”:

- έχει μία ουρά θεατών (**customers**)

Η κλάση “Ταμείο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά το ταμείο είναι άδειο.
- Στο ταμείο, ένας θεατής προστίθεται (**add**) στο τέλος της ουράς του.
- Στο ταμείο εξυπηρετείται (**serve**) ο πρώτος θεατής κάθε φορά, διαγράφοντάς τον από την ουρά.

Η κλάση “Bar”:

- έχει μία ουρά θεατών (**customers**)
- αποθηκεύει το τζίρο των πωλήσεων (**M**)

Η κλάση “Bar” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά το μπαρ είναι άδειο.
- Στο μπαρ, ένας θεατής προστίθεται (**add**) στο τέλος της ουράς του.
- Στο μπαρ εξυπηρετείται (**serve**) ο πρώτος θεατής κάθε φορά, συμβάλλοντας στο τζίρο με το ποσό αγοράς του, και διαγράφεται από την ουρά.

Η κλάση “Cinema”:

- έχει ένα ταμείο (**cashier**)
- έχει ένα μπαρ (**bar**)
- έχει μια σταθερά που αντιστοιχεί στο αντίτιμο του εισιτηρίου (**ticket_price**)
- αποθηκεύει τον τρέχοντα συνολικό τζίρο του (**M**)

Η κλάση “Cinema” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, ανατίθεται το αντίτιμο του εισιτηρίου του και ο συνολικός τζίρος είναι μηδέν.
- Κάνει εξυπηρέτηση, δημιουργώντας θεατή και προσθέτοντάς τον στο ταμείο. Κατόπιν, εξυπηρετεί το ταμείο, αυξάνοντας το συνολικό τζίρο κατά το αντίτιμο του εισιτηρίου. Αν ο θεατής που εξυπηρετήθηκε θέλει να αγοράσει κάτι από το μπαρ, τον προσθέτει στο μπαρ και τυχαία επιλέγει να εξυπηρετήσει το μπαρ ή όχι (**serve**)

Ο θεατής χαρακτηρίζεται από το ποσό αγοράς του από το μπαρ. Το ποσό αυτό δημιουργείται τυχαία κατά τη δημιουργία του θεατή. Αν δεν θέλει να αγοράσει κάτι από το μπαρ, το ποσό αυτό είναι μηδέν.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, κάνοντας παραδοχές στις προδιαγραφές, όπου (και αν) το θεωρείτε απαραίτητο, τεκμηριώνοντάς τις. *Σημείωση:* Θεωρείστε δεδομένο ένα τύπο ουράς αλλά δώστε τις προδιαγραφές του.

Σημείωση: Στα θέματα στα οποία σας ζητείται να βρείτε αποτελέσματα, όποτε αυτά επαναλαμβάνονται ή η αιτιολόγηση είναι ίδια με κάτι που ήδη έχει αιτιολογηθεί, απλά κάντε τη σχετική αναφορά, αν σας εξυπηρετεί.

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

Κ10:ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Εξετάσεις 10 Σεπτεμβρίου 2019

1. (α') Τόσο στη C++ όσο και στη Java διακρίνουμε τα μέλη κλάσεων σε στατικά (static) και μη στατικά. Σε τι αναφέρεται αυτή η διάκριση; Είναι το ίδιο κρίσιμα τα στατικά μέλη και για τις δύο γλώσσες; Γιατί;

- (β') Δίδονται τα παρακάτω προγράμματα C++ και Java:

Πρόγραμμα C++:

```
#include<iostream>
using namespace std;

class A{
    int data;
public:
    A(int i=10):data(i) {
        cout << "An A was just constructed with: " << data << endl;}
    void inc() {data++;}
    void print() {cout << data << endl;} };

class B : public A{
    int data;
public:
    B(int i=100):A(100),data(i) {
        cout << "A B was just constructed with: " << data << endl;}
    void inc() {data++;}
    void print() {A:: print(); cout << data << endl;} };

int main(){ B b; b.inc(); b.print();
            A* pab = new B(); pab->inc(); pab->print(); }
```

Πρόγραμμα Java:

```
public class Main{
    public static void main(String[] args){
        B.inc(); A.print_count(); B.print_count();
        B b = new B(); b.inc(); b.print_count();
        A a = new B(); a.inc(); a.print_count();}}

class A{
    private static int count;
    static {System.out.println("Initializing A ");}
    public static void inc() {count++;}
    public static void print_count() {System.out.println(count);} }

class B extends A{
    private static int count;
    static {System.out.println("Initializing B ");}
    public static void inc() {count++;}
    public static void print_count() {System.out.println(count);} }
```

Δώστε τα αποτελέσματα της εκτέλεσης των παραπάνω προγραμμάτων, αιτιολογώντας τα.

2. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++.

```
#include<iostream>
using namespace std;

class A{ int data;
public:
    A(int d=10) : data(d) {cout << "An A just constructed" << endl;}
    A(const A& a) : data(a.data) {cout << "An A just constructed by CP" << endl;}
    ~A(){cout << "An A to be destructed with: " << data << endl;}
    void inc() { data++;}
    int get() {return data;} };

class B{ A data1; A& data2; A* data3;
public:
    B(const A& d1, A& d2, const A& d3) : data1(d1), data2(d2){
        data3 = new A(d3); cout << "A B just constructed" << endl;}
    B(const B& b) : data1(b.data1), data2(b.data2){
        data3 = new A(*(b.data3)); cout << "A B just constructed by CP " << endl;}
    ~B(){cout << "A B to be destructed with: " << data1.get()
        << " and " << data2.get() << " and " << data3->get() << endl;
        delete data3; }
    B& operator=(const B& b)
        { cout<< "I am performing a B assignment " << endl;
          if (this != &b){
              delete data3;
              data1 = b.data1; data2 = b.data2; data3 = new A(*(b.data3));}
          return *this;}
    void inc() {data1.inc(); data2.inc(); data3->inc();} };

void fun(B b1, B& b2, B* pb3){
    b1.inc(); b2.inc(); pb3->inc();
    b2 = *pb3; b1 = b2;
    b1.inc(); b2.inc(); pb3->inc(); }

int main(){ A a1; A a2(100);
    B b1(a1, a2, a1); B b2(a2, a1, a2);
    fun(b1, b2, &b1); }
```

3. Ας υποθέσουμε ότι για την εκτίμηση της δυσκολίας (φόρτου) ενός ακαδημαϊκού εξαμήνου σε ένα πανεπιστημιακό τμήμα ισχύουν τα παρακάτω.

Το εξάμηνο αποτελείται από 5 (πέντε) μαθήματα. Το κάθε μάθημα περιλαμβάνει θεωρία (διδασκαλία με τη μορφή παραδόσεων και φροντιστηρίων) αλλά μπορεί να περιλαμβάνει και εργαστήριο ή/και ένα πρακτικό μέρος. Η θεωρία χαρακτηρίζεται από τον αριθμό των ωρών ανά εβδομάδα που διαρκεί η διδασκαλία. Το ίδιο και το εργαστήριο. Το εργαστήριο όμως μπορεί να έχει υποχρεωτική παρακολούθηση ή όχι. Το πρακτικό μέρος αποτελείται από μια σειρά από παραδοτέες ασκήσεις. Η κάθε άσκηση αποτελείται από ένα κείμενο που αντιστοιχεί στην εκφώνησή της.

Ο υπολογισμός της δυσκολίας (`evaluate_load`) του κάθε εξαμήνου προκύπτει από την συνολική δυσκολία των μαθημάτων του.

Ο υπολογισμός της δυσκολίας (`evaluate_load`) ενός μαθήματος προκύπτει από το άθροισμα της δυσκολίας της θεωρίας και της δυσκολίας του εργαστηρίου του πολλαπλασιασμένο με ένα συντελεστή (σταθερά του μαθήματος). Επιπλέον, προστίθεται και η δυσκολία του πρακτικού μέρους.

Η δυσκολία της θεωρίας (`evaluate_load`) προκύπτει από το πλήθος των ωρών της. Η δυσκολία του εργαστηρίου (`evaluate_load`) προκύπτει από το πλήθος των ωρών του. Αν το εργαστήριο δεν είναι υποχρεωτικό, μειώνεται στο μισό.

Ο υπολογισμός της δυσκολίας (`evaluate_load`) του πρακτικού μέρους προκύπτει από το άθροισμα της δυσκολίας των επί μέρους ασκήσεων. Η δυσκολία μιας άσκησης είναι σταθερά της άσκησης.

Το εξάμηνο αρχικοποιείται με την πληροφορία των μαθημάτων που περιλαμβάνει. Ένα μάθημα αρχικοποιείται με τις πληροφορίες για τη θεωρία του, για το εργαστήριο και για το πρακτικό μέρος. Η θεωρία αρχικοποιείται με το πλήθος των ωρών διδασκαλίας της. Το εργαστήριο αρχικοποιείται με το πλήθος των ωρών διδασκαλίας του και την ένδειξη αν είναι υποχρεωτικό ή όχι. Το πρακτικό μέρος αρχικοποιείται με τις ασκήσεις που το αποτελούν. Μια τέτοια άσκηση αρχικοποιείται με το κείμενο της εκφώνησής της. Κατά την αρχικοποίηση παρέχονται οι πληροφορίες και για τις σταθερές, όπου χρειάζεται.

Τόσο για το εξάμηνο αλλά και για τα μαθήματα, τη θεωρία, το εργαστήριο και το πρακτικό μέρος να παρέχονται συναρτήσεις εκτύπωσης (`print`) που να εκτυπώνουν τις πληροφορίες τους.

Να υλοποιήσετε την παραπάνω πληροφορία μέσω κλάσεων C++.

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10:ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Εξετάσεις 8 Σεπτεμβρίου 2020

1. (α') Αναφέρετε δύο βασικές λειτουργικότητες που πρέπει να παρέχει μια γλώσσα προγραμματισμού στον προγραμματιστή προκειμένου να μπορεί να θεωρηθεί γλώσσα αντικειμενοστραφούς προγραμματισμού.
- (β') Αιτιολογώντας την απάντησή σας —με όποιον τρόπο σας εξυπηρετεί—, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++.

```
#include<iostream>
using namespace std;
class A{ int data;
public:
    A(): data(10) { cout << "A new A has been constructed" << endl; }
    virtual ~A() { cout << " An A will be destructed with data: " << data << endl; }
    virtual void f1() { cout << "A::f1() is executing " << endl; data++; }
    void f2() { cout << "A::f2() is executing " << endl; f1(); } };

class B: public A{ int data;
public:
    B(): data(100) { cout << "A new B has been constructed" << endl; }
    ~B() { cout << " A B will be destructed with data: " << data << endl; }
    void f1() { cout << "B::f1() is executing " << endl; data++; }
    virtual void f2() { cout << "B::f2() is executing " << endl; f1(); } };

class C: public B{ int data;
public:
    C(): data(1000) { cout << "A new C has been constructed" << endl; }
    ~C() { cout << " A C will be destructed with data: " << data << endl; }
    void f1() { cout << "C::f1() is executing " << endl; data++; }
    void f2() { cout << "C::f2() is executing " << endl; f1(); } };

int main(){

    A* pa = new A(); pa->f1(); pa->f2(); delete pa;

    pa = new B(); pa->f1(); pa->f2(); delete pa;

    pa = new C(); pa->f1(); pa->f2(); delete pa; }
```

2. Θεωρήστε τον παρακάτω κόσμο ζωτικών που ευλογούν και, σε κάποιες περιπτώσεις, καταριούνται το ένα το άλλο. Τα ζωτικά διακρίνονται σε καλά ζωτικά και κακά ζωτικά.

Κάθε ζωτικό (creature)

- έχει ένα βαθμό ευεξίας (`happiness_degree`)
- μπορεί να ευλογήσει (`spell`) ένα άλλο. Σε αυτή την περίπτωση, ο βαθμός ευεξίας του ζωτικού που ευλογείται αυξάνεται κατά μια μονάδα.
- υφίσταται κατάρα (`gets_curse`). Σε αυτή την περίπτωση, ο βαθμός ευεξίας του μειώνεται κατά μία μονάδα. Αν ο βαθμός ευεξίας ενός ζωτικού μηδενιστεί, να τυπώνεται το μήνυμα "What a miserable world"

Αν ο βαθμός ευεξίας ενός ζωτικού γίνει ίσος με μια σταθερά μέγιστου βαθμού, κοινή για όλα τα ζωτικά (`MAX_HAPPINESS`), δεν μπορεί να τροποποιηθεί —ούτε να μειωθεί ούτε να αυξηθεί—.

Κάθε καλό ζωτικό (`good_creature`)

- έχει επιπλέον ένα μετρητή βαθμού καλοσύνης (`kindness_degree`)
- όταν ευλογεί (`spell`), επιπλέον της συμπεριφοράς του ως ζωτικό, αυξάνεται κατά μία μονάδα και ο βαθμός καλοσύνης του. Για να συμβούν αυτά, πρέπει ο βαθμός ευεξίας του να είναι πάνω από μηδέν.

Κάθε κακό ζωτικό (`bad_creature`)

- έχει επιπλέον ένα μετρητή βαθμού κακίας (`badness_degree`)
- όταν ευλογεί (`spell`), επιπλέον της συμπεριφοράς του ως ζωτικό, μειώνεται κατά μία μονάδα και ο βαθμός κακίας του.
- μπορεί να καταραστεί (`curse`) ένα άλλο ζωτικό, οτιδήποτε και να είναι αυτό και, τότε, αυξάνεται κατά μία μονάδα ο βαθμός κακίας του. Το ζωτικό στο οποίο απευθύνεται η κατάρα, υφίσταται κατάρα (`gets_curse`). Για να συμβούν αυτά, πρέπει ο βαθμός κακίας του κακού ζωτικού να είναι πάνω από μηδέν και ο βαθμός ευεξίας του μικρότερος από τη σταθερά μέγιστου βαθμού ευεξίας.

Υλοποιήστε σε C++ τις κλάσεις που αναπαριστούν τον παραπάνω κόσμο. Οι αρχικές τιμές δεδομένων των αντικειμένων των κλάσεων να παίρνουν τιμές που να δίδονται κατά τη δημιουργία των αντικειμένων. Σημείωση: Μπορείτε να κάνετε δικές σας παραδοχές για τον κόσμο αυτό αρκεί να μην έρχονται σε αντίφαση με την παραπάνω περιγραφή.

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
Κ10: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ
Εξετάσεις, Τετάρτη 14 Σεπτεμβρίου 2022

1. (α') Τι εννοούμε με τον όρο “εμβέλεια” ονομάτων; Αναφέρατε δύο διαφορετικούς τρόπους ορισμού εμβέλειας στη C++.
- (β') Αιτιολογώντας την απάντησή σας, δώστε την εκτύπωση του παρακάτω προγράμματος C++.

```
#include <iostream>
using namespace std;

class Fat{
    int data;
public:
    Fat(int i = 10) : data(i) {cout << "Constructing a Fat " << endl;}
    ~Fat() {cout << "Destructing a Fat with data: " << data << endl;}
    virtual int inc() { return data++; }
};

class SubFat : public Fat{
    int data;
public:
    SubFat(int i = 20) : data(i) {cout << "Constructing a SubFat" << endl;}
    ~SubFat() {cout << "Destructing a SubFat with data: " << data << endl;}
    int inc() { return data++; }
};

int main()
{
    SubFat sf(50); Fat* pf = &sf;

    cout << sf.inc() << endl;
    cout << pf->inc() << endl;

    return 0;
}
```

2. Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση της φόρτωσης οχηματαγωγών πλοίων. Προκειμένου να γίνει η υλοποίηση αυτή θεωρούμε τις κλάσεις: “επιβατικό αυτοκίνητο” (Car), “φορτηγό αυτοκίνητο” (Truck) και “οχηματαγωγό πλοίο” (Ferry_boat).

Η κλάση “επιβατικό αυτοκίνητο”:

- έχει έναν μοναδικό αριθμό κυκλοφορίας (id)
- έχει ένα πλοίο στο οποίο θα επιβιβαστεί (boat)
- έχει μια ένδειξη αν έχει επιβιβαστεί ή όχι (boarded)
- έχει ένα πλήθος επιβατών που μεταφέρει (passengers).

Η κλάση “επιβατικό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά καταχωρείται ο αριθμός κυκλοφορίας του, το πλοίο που θα επιβιβαστεί καθώς και το πλήθος των επιβατών που μεταφέρει. Θεωρούμε ότι αρχικά δεν έχει επιβιβαστεί.
- Επιστρέφεται το πλήθος των επιβατών που μεταφέρει (get_passengers).
- Το αυτοκίνητο επιβιβάζεται στο πλοίο, πραγματοποιώντας στο πλοίο επιβίβαση επιβατικού αυτοκινήτου. Αν είναι επιτυχής, αλλάζει η ένδειξη ότι έχει επιβιβαστεί (board).

Η κλάση “φορτηγό αυτοκίνητο”:

- έχει έναν μοναδικό αριθμό κυκλοφορίας (`id`)
- έχει ένα πλοίο στο οποίο θα επιβιβαστεί (`boat`)
- έχει μια ένδειξη αν έχει επιβιβαστεί ή όχι (`boarded`)

Η κλάση “φορτηγό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά καταχωρείται ο αριθμός κυκλοφορίας του και το πλοίο που θα επιβιβαστεί -θεωρούμε ότι έχει έναν μόνο επιβάτη, τον οδηγό, και δεν χρειάζεται να γίνει κάποια ενέργεια αρχικοποίησης γι'αυτόν. Θεωρούμε ότι αρχικά δεν έχει επιβιβαστεί.
- Το φορτηγό επιβιβάζεται στο πλοίο, πραγματοποιώντας στο πλοίο επιβίβαση φορτηγού. Αν είναι επιτυχής, αλλάζει η ένδειξη ότι έχει επιβιβαστεί (`board`).

Η κλάση “οχηματαγωγό πλοίο”:

- έχει έναν μοναδικό αριθμό νηολογίου (`id`)
- έχει έναν μετρητή επιβιβασμένων επιβατών (`no_passengers`)
- έχει ένα μέγιστο αριθμό επιβατών (`max_no_passengers`)
- έχει έναν μετρητή επιβιβασμένων επιβατικών αυτοκινήτων (`no_cars`)
- έχει ένα μέγιστο αριθμό επιβατικών αυτοκινήτων (`max_no_cars`)
- έχει έναν μετρητή επιβιβασμένων φορτηγών (`no_trucks`)
- έχει ένα μέγιστο αριθμό φορτηγών (`max_no_trucks`)

Η κλάση “οχηματαγωγό πλοίο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά καταχωρείται ο αριθμός νηολογίου του, ο μέγιστος αριθμός επιβατών και οι μέγιστοι αριθμοί οχημάτων. Αρχικά, το πλοίο είναι άδειο.
- Επιβιβάζεται ένα πλήθος επιβατών, εάν με την επιβίβασή τους δεν υπερβαίνουμε το μέγιστο αριθμό επιβατών του πλοίου. Τότε, αυξάνεται ο μετρητής επιβιβασμένων επιβατών του πλοίου κατά το πλήθος τους (`board`).
- Γίνεται επιβίβαση επιβατικού αυτοκινήτου, εάν δεν υπερβαίνουμε το μέγιστο αριθμό επιβατικών αυτοκινήτων και δεν υπερβαίνουμε το μέγιστο αριθμό επιβατών λόγω των επιβατών του. Στην περίπτωση που τα παραπάνω δεν παραβιάζονται, αυξάνουμε τον μετρητή των επιβιβασμένων αυτοκινήτων κατά ένα και τον μετρητή των επιβιβασμένων επιβατών κατά το πλήθος των επιβατών που μεταφέρει (`board`).
- Γίνεται επιβίβαση φορτηγού αυτοκινήτου, εάν δεν υπερβαίνουμε το μέγιστο αριθμό φορτηγών αυτοκινήτων και δεν υπερβαίνουμε το μέγιστο αριθμό επιβατών λόγω του οδηγού του. Στην περίπτωση που τα παραπάνω δεν παραβιάζονται, αυξάνουμε τον μετρητή των επιβιβασμένων φορτηγών αυτοκινήτων καθώς και τον μετρητή των επιβιβασμένων επιβατών κατά ένα (`board`).

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, ορίζοντας νέες όπου δείχνει ενδεικνυόμενο. Δεν ζητείται υλοποίηση συνάρτησης `main`.

ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ	
Ιανουάριος 2025	A.M.
Επώνυμο:	Όνομα:

ΔΙΑΡΚΕΙΑ ΕΞΕΤΑΣΗΣ : 3 ώρες

Θέμα 1 (2.5 μονάδες) Θεωρούμε τα παρακάτω πρόγραμμα. Τι τυπώνονται κατά την εκτέλεση του. Δικαιολογείστε τις απαντήσεις σας.

α) (μονάδα 1)

```

1  #include <iostream>
2  using namespace std;
3
4  class A{
5      int data;
6      public:
7      A(int dt=0):data(dt){
8          cout<<"Creating an A"<<endl;
9      }
10     A( A& a):data(a.data){
11         cout<<"Creating an A2"<<endl;
12     }
13     A& operator=(const A&)
14     {
15         cout<<"do a3"<<endl;
16         return *this;
17     }
18     ~A(){
19         cout<<"Destroying an A with data " <<endl;
20         cout<<data<<endl;
21     }
22     void put(int data){
23         data=data;
24     }
25 };
26
27
28 int main()
29 {
30     A a1; A a2(10); A a3(a2);
31     A a4=a2; A& a5=a1;a5=a2;
32     a1.put(6);a3=a1;
33     return 0;
34 }
```

β) (μονάδα 0,5)

```

1  #include <iostream>
2  using namespace std;
3
4  class base {
5      public:
6      base() {
7          cout << "Constructing base\n";
8      }
9      ~base() {
10         cout << "Destructing base\n";
11     }
12 };
13
14 class derived : public base {
15     public:
16     derived() {
17         cout << "Constructing derived\n";
18     }
19     ~derived() {
20         cout << "Destructing derived\n";
21     }
22 };
23
24 int main() {
25     derived *d = new derived();
26     base *b = d;
27     delete b;
28     return 0;
29 }
```


γ) (μονάδα 0,5)

```

1  #include <iostream>
2  using namespace std;
3
4  class one {
5  public:
6      virtual void f1() {
7          cout << "a";
8          f2();
9      }
10     void f2() {
11         cout << "b";
12     }
13 };
14
15 class two : public one {
16 public:
17     virtual void f1() {
18         cout << "c";
19         f2();
20     }
21     void f2() {
22         cout << "a";
23     }
24 };
25
26 int main() {
27     one* p;
28     p=new one;p->f1();
29     p=new two;p->f1();
30     two *q;
31     q=new two;q->f1();
32     return 0;
33 }

```

δ) (μονάδα 0,5)

```

4  class A {
5  protected:
6      int dataA;
7  public:
8      A() : dataA(0) {}
9      A(int d) : dataA(d) {
10         if (d < 0) throw "A";
11     }
12     int get() { return dataA; }
13 };
14
15 class B : public A {
16 protected:
17     int dataB;
18 public:
19     B() : dataB(1) {}
20     B(int d) : dataB(d) {
21         if (d == 0) throw dataB;
22     }
23     int get() { return dataB; }
24 };
25
26 int main() {
27     try {
28         A a(2);
29         cout << a.get();
30         B b1;
31         cout << b1.get();
32         B b2(0);
33         cout << b2.get();
34     } catch (const char* e) {
35         cout << e;
36     } catch (int i) {
37         cout << i;
38     }
39     return 0;
40 }

```

Θέμα 2 (1,5 μονάδες)

α) Στο πρόγραμμα του ερωτήματος 1γ, φτιάξτε τα virtual table που υπάρχουν (μονάδα 1)

β) Θέλουμε να ορίσουμε μία κλάση B που κληρονομεί την A, αλλά θέλω να διασφαλίσω ότι ο χρήστης δεν θα μπορεί να ορίσει αντικείμενα της κλάσης A. Δώστε παράδειγμα σύμφωνα με το οποίο μπορεί να γίνει αυτό. (μονάδα 0,5)

Θέμα 3 (1 μονάδες)

Υπάρχουν λάθη στα παρακάτω προγράμματα; Αν ναι πείτε σε ποια γραμμή υπάρχει πρόβλημα και αιτιολογείστε το.

α)	<pre>1 #include <iostream> 2 using namespace std; 3 4 class A { 5 void priv() {} 6 protected: 7 void prot() {} 8 public: 9 void pub() {} 10 }; 11 12 class B : public A { 13 A a; 14 B* pb; 15 public: 16 void f() { 17 priv(); 18 pb->prot(); 19 a.prot(); 20 } 21 }; 22 23 int main() { 24 B b; 25 b.prot(); 26 return 0; 27 }</pre>	β)	<pre>1 #include <iostream> 2 using namespace std; 3 4 class A { 5 private: 6 int p; 7 public: 8 A(int a) : p(a) {} 9 }; 10 11 class B : public A { 12 public: 13 B(int v) { 14 cout << v << endl; 15 } 16 void show(int v) const { 17 cout << p+v << endl; 18 } 19 }; 20 21 int main() { 22 B b(10); 23 A& r=b; 24 r.show(20); 25 return 0; 26 }</pre>
----	---	----	--

Θέμα 4 Java(2 μονάδες)

α) Έστω ένα Interface Person και θέλετε να ορίσετε μία κλάση Student από αυτό. Πώς θα γράφατε τον ορισμό της κλάσης Student; Ποια η διαφορά μεταξύ Interface και κλάσης;

β) Θέλετε να φτιάξετε σε java μια κλάση C3 που να κληρονομεί από την C1 και C2. Πως θα κάνατε τους ορισμούς σας;

γ) Γράψτε το Javadoc για τη συνάρτηση `public int f(int a, int b)` που υπολογίζει το άθροισμα δύο ακέραιων αριθμών. Φροντίστε να περιγράψετε το σκοπό της συνάρτησης, τις παραμέτρους της και την τιμή που επιστρέφει.

δ) Τι θα εμφανιστεί;

```
1 class Base {
2     int value;
3
4     Base(int value) {
5         this.value = value;
6         System.out.println("Constructor 1" + value);
7     }
8 }
9
10 class Derived extends Base {
11     int multiplier;
12
13     Derived() {
14         this.multiplier = 1;
15         System.out.println("Constructor 2");
16     }
17
18     Derived(int multiplier) {
19         super(multiplier * 2);
20         this.multiplier = multiplier;
21         System.out.println("Constructor 3 " + multiplier);
22     }
23 }
24
25 public class Main {
26     public static void main(String[] args) {
27         Derived obj1 = new Derived();
28         Derived obj2 = new Derived(5);
29     }
30 }
```

Θέμα 5(3 μονάδες)

Να ορίσετε την κλάση **Fruit** με ιδιωτικά πεδία το όνομα του φρούτου και τον αριθμό των θερμίδων του (πχ cal). Η κλάση να περιέχει κατάλληλο κατασκευαστή που να αρχικοποιεί τα πεδία της. Από την κλάση **Fruit** να παράγεται με δημόσια πρόσβαση η κλάση **Apple** με ιδιωτικό πεδίο τον αριθμό των αντιοξειδωτικών που περιέχει (πχ antiox). Η κλάση να περιέχει κατάλληλο κατασκευαστή που να αρχικοποιεί το πεδίο της και ο οποίος να καλεί τον κατασκευαστή της **Fruit**. Να ορίσετε την συνάρτηση **f()** η οποία να δέχεται σαν όρισμα μία αναφορά σε ένα αντικείμενο και μία ακέραια τιμή και να τα προσθέτει. Να προσθέσετε ότι χρειάζεται για να λειτουργεί το παρακάτω πρόγραμμα.

```
1 int main() {
2     Fruit fr("F1",10);
3     Apple ap(20,"F2",30);
4     f(fr,5);
5     f(ap,5);
6     return 0;
7 }
8
9 void f(__ &p, int num){
10     cout<< p.num << '\n';
11 }
```

- Στη γραμμή 4 εμφανίζει 15, γιατί στο 5 προσθέτει το cal που είναι 10
- Στη γραμμή 5 εμφανίζει 25, γιατί στο 5 προσθέτει την τιμή του antiox που είναι 20

ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ	
Σεπτέμβριος 2024	A.M.
Επώνυμο:	Όνομα:

ΔΙΑΡΚΕΙΑ ΕΞΕΤΑΣΗΣ : 3 ώρες

Θέμα 1 (2 μονάδες) Θεωρούμε τα παρακάτω πρόγραμμα. Τι τυπώνονται κατά την εκτέλεση τους; Δικαιολογείστε τις απαντήσεις σας.

α)

```

1  #include <iostream>
2  using namespace std;
3
4  class A{
5      public:
6          A() {cout<<"A";}
7          ~A() {cout<<"a";}
8          void o() {cout<<"#";}
9  };
10 class B:public A{
11     public:
12         B() {cout<<"B";}
13         ~B() {cout<<"b";}
14         void o() {cout<<"@";}
15 };
16
17 int main(){
18     B b;
19     A *c = &b;
20     c->o();
21     b.o();
22 }
```

β)

```

1  #include <iostream>
2  using namespace std;
3
4  class A{
5      public:
6          void f1(){ cout<<"1"; f2();}
7          virtual void f2() {cout<<"2";}
8  };
9
10 class B: public A{
11     public:
12         void f1(){ cout<<"3"; f2();}
13         void f2() override {cout<<"4";}
14 };
15
16 int main(){
17     A *p = new A;
18     B *q = new B;
19     p->f1();
20     q->f1();
21     q->f2();
22     q->f1();
23 }
```

γ)

```

1  #include <iostream>
2  using namespace std;
3
4  class A{
5      static int x;
6      int y;
7      public:
8          static void setX(int a){ x--a;}
9          A(int b): y(b) {}
10         void status() const {
11             cout<<" " << x <<" "<<y;}
12     };
13     int A::x = 7;
14
15 int main(){
16     A a1(0);
17     a1.status();
18     a1.setX(1);
19     a1.setX(2);
20     a1.status();
21     A a2(3);
22     a2.status();
23     a1.setX(4);
24     a2.status();
25 }
```

δ)

```

1  #include <iostream>
2  using namespace std;
3
4  class A{
5      protected:
6          int dataA;
7      public:
8          A() : dataA(0) {}
9          A(int d) : dataA(d){throw "A";}
10         int get() {return dataA;}
11     };
12 class B:public A{
13     protected:
14         int dataB;
15     public:
16         B() {dataB = 1;}
17         B(int d) : dataB(d){throw dataB;}
18         int get() {return dataB;}
19     };
20
21 int main(){
22     try{
23         A a;
24         cout << a.get();
25         B b1;
26         cout << b1.get();
27         B b2(1);
28         cout << b2.get();
29         B b3(2);
30         cout << b3.get();
31     }
32     catch (const char *a) {cout << a;}
33     catch (const int i) {cout << i;}
34 }
```


Θέμα 2 (2.5 μονάδες)

Σε κάθε ένα από τα παρακάτω ερωτήματα να επιλέξετε μία απάντηση και να την δικαιολογήσετε. Εάν δεν την δικαιολογήσετε η απάντησή σας δεν θα γίνει δεκτή.

i) Θεωρούμε το παρακάτω πρόγραμμα. Τι ισχύει για το παρακάτω πρόγραμμα; (0.5 μονάδες)

```

1 class A{
2     double x, y;
3     public:
4         A(double x1, double y1) : x(x1), y(y1){};
5 };
6 bool operator== (A a1, A a2){
7     return ((a1.x==a2.x) && (a1.y==a2.y));
8 }
9
10 int main(){
11     A a1(1,2), a2(1,2);
12     cout<< ((a1==a2)? "equal" : "different");
13 }
```

- α) Το πρόγραμμα λειτουργεί ως έχει
- β) Ο τελεστής == πρέπει να δηλωθεί ως μέλος της κλάσης A
- γ) Ο τελεστής == πρέπει να δηλωθεί ως friend της κλάσης A
- δ) οποιοδήποτε από το β ή γ

ii) (0.2 μονάδες)

Έχουμε ορίσει μία κλάση τα αντικείμενα της οποίας παριστάνουν ημερομηνίες και θέλουμε να υλοποιήσουμε τη σύγκριση ημερομηνιών ορίζοντας κατάλληλα τον operator<

- α) Μπορούμε να τον ορίσουμε ως φίλη (friend) συνάρτηση με δύο παραμέτρους
- β) Μπορούμε να τον ορίσουμε ως μέθοδο της κλάσης με μία παράμετρο
- γ) Μπορούμε να τον ορίσουμε ως μέθοδο της κλάσης με δύο παραμέτρους
- δ) Μπορούμε να κάνουμε οποιαδήποτε από τα Α ή Β

iii) (0.15 μονάδες)

Θέλουμε μία αποδοτική δομή δεδομένων για να αποθηκεύσουμε ένα πολύ μεγάλο πλήθος μοναδικών strings τα οποία στη συνέχεια να αναζητούμε και να εμφανίζουμε ταξινομημένα. Είναι καλή ιδέα να χρησιμοποιήσουμε STL και συγκεκριμένα

- α) map
- β) multimap
- γ) set
- δ) multiset

iv) (0.15 μονάδες)

Θέλουμε να υλοποιήσουμε την εκτύπωση των αντικειμένων μίας δικής μας κλάσης ορίζοντας κατάλληλα τον operator <<

- α) Μπορούμε να τον ορίσουμε ως μέθοδο της κλάσης μας με μία παράμετρο
- β) Μπορούμε να τον ορίσουμε ως μέθοδο της κλάσης μας με δύο παραμέτρους
- γ) Μπορούμε να τον ορίσουμε ως φιλική (friend) συνάρτηση με δύο παραμέτρους
- δ) Μπορούμε να κάνουμε οποιοδήποτε από το Α ή Γ

v) Τι τυπώνει το παρακάτω πρόγραμμα (0.5 μονάδες)

```

1 int f(int a[], int b, int t){
2     if (b>t) return a[b];
3     int mid = (b+t)/2;
4     int x1 = what(a,b,mid);
5     int x2 = what(a,mid+1,t);
6     if(x1<x2) return x1;
7     else return x2;
8 }
9 int main(){
10     int A[]={3,7,8,11,13,5,16,9,12};
11     cout<< f(A,0,6) << " " << f(A,4,7)<<endl;
12 }

```

α) 11 13

β) 3 5

γ) 16 16

δ) κανένα από τα προηγούμενα

vi) Τι τυπώνει το παρακάτω πρόγραμμα (0.5 μονάδες)

```

1 int k = 20;
2 void f(int &n){
3     n++;
4     cout<< n*k/2 << " ";
5 }
6 int main(){
7     p(k);
8     cout<< k << endl;
9 }

```

α) 21020

β) 21021

γ) 22020

δ) 22021

vii) Τι τυπώνει το παρακάτω πρόγραμμα (0.5 μονάδες)

```

1 int x[] = {0, 5, 10, 15};
2 int main(){
3     int *p = x, *q = x;
4     p++; ++*p; q+=3; *q+=2;
5     cout<<x[1]<<x[2]<<x[3]<<*p<<*q<<endl;
6 }

```

α) 61017617

β) 51017517

γ) 61015615

δ) κανένα από τα προηγούμενα

Θέμα 3(3 μονάδες)

α) Να ορίσετε την κλάση **Product** με προστατευμένο πεδίο τον κωδικό **code** του προϊόντος. Από την κλάση **Product** να παράγεται με δημόσια πρόσβαση η κλάση **Book** με ιδιωτικό πεδίο τον τίτλο **title** του βιβλίου. Κάθε κλάση να περιέχει την δική της έκδοση της **show()**, η οποία να εμφανίζει την τιμή του μέλους της. Να ορίσετε σωστά την συνάρτηση **f()** η οποία να δέχεται σαν ορίσματα δύο δείκτες σε αντικείμενα και να εμφανίζει τις τιμές των μελών τους. Προσθέστε στις κλάσεις κατάλληλες συναρτήσεις ώστε να λειτουργεί το παρακάτω πρόγραμμα.

(1 μονάδα)


```

1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      Product prod(100);
6      Book b("C++",200);
7      .....
8  }
9
10 void f(..... *p1,..... *p2){
11     p1->show();
12     p2->show();
13 }

```

- Στη γραμμή 5 το code του prod γίνεται 100
- Στη γραμμή 6 το code του b γίνεται 200 και το title του γίνεται C++
- Στη γραμμή 7 και για όσες γραμμές χρειαστείτε, δηλώστε ένα δείκτη p1 που να δείχνει στο prod, ένα δείκτη p2 που να δείχνει στο b και μετά να καλέστε την f. Να αιτιολογήσετε την επιλογή των δεικτών
- Στη γραμμή 11 να εμφανίζεται το code του prod
- Στη γραμμή 12 να εμφανίζεται το title του b

β) Να ορίσετε την κλάση **Library** που θα περιέχει ένα πίνακα **books** από βιβλία της κλάσης **Book** που ορίστηκε στο ερώτημα α. Προσθέστε στην κλάση ότι χρειάζεται ώστε να λειτουργεί σωστά το παρακάτω πρόγραμμα. (2 μονάδες)

```

1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      Library lib1(100), lib2(200);
6      Library lib3=lib1;
7      Book b("C++",200);
8      lib1 = lib1 + b;
9      lib2 = lib1;
10 }

```

- Στη γραμμή 5 ο πίνακας books του lib1 γίνεται μεγέθους 100, ενώ του lib2 γίνεται μεγέθους 200
- Στη γραμμή 8 το b προστίθεται στον πίνακα books του lib1 και μπαίνει στην πρώτη θέση του πίνακα

Θέμα 4 (2,5 μονάδες)

Σε ένα κατάστημα οι ταμειακές μηχανές καταχωρούν σε ένα αρχείο "data.txt" όνομα προϊόντος και τιμή που αγοράστηκε, με αποτέλεσμα στο ίδιο αρχείο το ίδιο προϊόν να υπάρχει πολλές φορές. Να γράψετε ένα πρόγραμμα που θα διαβάζει το αρχείο και θα εμφανίζει στην οθόνη το κάθε προϊόν μία φορά, πόσες φορές υπάρχει επί την τιμή της μονάδας καθώς και το σύνολο του. Στο τέλος, θα υπάρχει και ο συνολικός λογαριασμός. Το πρόγραμμα σας θα πρέπει να περιέχει μία κλάση **cashRegister**, η οποία να χρησιμοποιεί κατάλληλη κλάση της STL για την καταχώρηση κάθε γραμμής του τελικού λογαριασμού, καθώς και μεθόδους για το σκανάρισμα είδους και την εμφάνιση του τελικού λογαριασμού. Στη συνέχεια, δίνεται ένα παράδειγμα του αρχείου εισόδου και της αντίστοιχης εμφάνισης στην οθόνη.

<u>"data.txt"</u>	<u>Οθόνη</u>
Chair 24	Chair 4 x 24 = 96
Chair 24	Desk 1 x 80 = 80
Desk 80	Bed 2 x 140 = 640
Bed 140	Table 3 x 100 = 300
Chair 24	Sofa 1 x 300 = 300
Table 100	Carpet 1 x 184 = 184
Sofa 300	Total: 1600
Chair 24	
Bed 500	
Table 100	
Carpet 184	